TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN HỌC PHẦN
CÔNG NGHỆ PHẦN MỀM

ĐỀ TÀI:

HỆ THỐNG ĐẶT CÀ PHÊ TRỰC TUYẾN BREWLITE

Giảng viên hướng dẫn: Từ Lãng Phiêu

Sinh viên thực hiện:

Phạm Minh Chiến – 3124410027	(Trưởng nhóm – Auth)
Đinh Nhật Lan – 3124410182	(Thành viên – Catalog)
Vũ Hoàng Huy – 3124410125	(Thành viên – Detail)
Quách Tuấn Hào – 3124560029	(Thành viên – Cart)
Tôn Thất Hạnh – 3124410079	(Thành viên – Order)
Nguyễn Trần Gia Kiệt – 3124410176	(Thành viên – Payment)
Nguyễn Quang Huy – 3124560034	(Thành viên – History)

Lớp / Khóa: **[TODO: DCT12...]** – **[TODO: 2024 – 2028]**

TP. HỒ CHÍ MINH, THÁNG 10 NĂM 2026

MỤC LỤC

1 GIỚI THIỆU DỰ ÁN **1**
- 1.1 Tổng quan đề tài và Product Vision . . . 1
- 1.2 Phạm vi hệ thống (MVP Scope) . . . 1
- 1.3 Ngăn xếp công nghệ (Technology Stack) . . . 2
- 1.4 Quy trình phát triển phần mềm . . . 2

2 YÊU CẦU HỆ THỐNG **4**
- 2.1 Các tác nhân hệ thống (Stakeholders & Actors) . . . 4
- 2.2 Danh mục yêu cầu chức năng (Functional Requirements) . . . 4
- 2.3 Danh mục yêu cầu phi chức năng (Non-Functional Requirements) . . . 4
- 2.4 User Story tiêu biểu (US-05: Đặt hàng và áp dụng ưu đãi) . . . 6

3 KIẾN TRÚC HỆ THỐNG **8**
- 3.1 Kiến trúc thành phần (Component Architecture) . . . 8
- 3.2 Kiến trúc triển khai với Docker (Deployment Architecture) . . . 9
- 3.3 Sơ đồ lớp Backend (Backend Class Diagram) . . . 10
- 3.4 Luồng tương tác nghiệp vụ tổng thể (Overall Business Flow) . . . 11

4 THIẾT KẾ CƠ SỞ DỮ LIỆU **13**
- 4.1 Sơ đồ Thực thể Liên kết (ERD) toàn hệ thống . . . 13
- 4.2 Chi tiết 12 thực thể trong Cơ sở dữ liệu . . . 15
- 4.3 Ràng buộc toàn vẹn và Chiến lược đánh chỉ mục . . . 16
- 4.4 Thiết kế CSDL hỗ trợ nghiệp vụ nâng cao . . . 16

5 THIẾT KẾ CHI TIẾT CÁC PHÂN HỆ CHỨC NĂNG **18**
- 5.1 Phân hệ Xác thực Người dùng (Auth) . . . 18
  - 5.1.1 Đặc tả Use Case . . . 18
  - 5.1.2 Sơ đồ Hoạt động (Activity Diagram) . . . 18
  - 5.1.3 Sơ đồ Tuần tự (Sequence Diagram) . . . 18
  - 5.1.4 Hợp đồng API (API Contract) . . . 18
  - 5.1.5 Ánh xạ Cơ sở dữ liệu (Database Mapping) . . . 18
  - 5.1.6 Kịch bản Kiểm thử (Test Cases) . . . 22
  - 5.1.7 Giao diện Người dùng . . . 22
- 5.2 Phân hệ Danh mục Sản phẩm (Product Catalog) . . . 23
  - 5.2.1 Đặc tả Use Case . . . 23
  - 5.2.2 Sơ đồ Hoạt động (Activity Diagram) . . . 23
  - 5.2.3 Sơ đồ Tuần tự (Sequence Diagram) . . . 23
  - 5.2.4 Hợp đồng API (API Contract) . . . 23
  - 5.2.5 Ánh xạ Cơ sở dữ liệu (Database Mapping) . . . 23
  - 5.2.6 Kịch bản Kiểm thử (Test Cases) . . . 26
  - 5.2.7 Giao diện Người dùng . . . 26
- 5.3 Phân hệ Chi tiết Sản phẩm (Product Detail) . . . 27
  - 5.3.1 Đặc tả Use Case . . . 27
  - 5.3.2 Sơ đồ Hoạt động (Activity Diagram) . . . 27
  - 5.3.3 Sơ đồ Tuần tự (Sequence Diagram) . . . 27
  - 5.3.4 Hợp đồng API (API Contract) . . . 27

5.3.5 Ánh xạ Cơ sở dữ liệu (Database Mapping) . . . 27
5.3.6 Kịch bản Kiểm thử (Test Cases) . . . 30
5.3.7 Giao diện Người dùng . . . 30
5.4 Phân hệ Giỏ hàng (Cart) . . . 30
5.4.1 Đặc tả Use Case . . . 30
5.4.2 Sơ đồ Hoạt động (Activity Diagram) . . . 30
5.4.3 Sơ đồ Tuần tự (Sequence Diagram) . . . 33
5.4.4 Hợp đồng API và Ánh xạ Cơ sở dữ liệu . . . 33
5.4.5 Kịch bản Kiểm thử (Test Cases) . . . 34
5.4.6 Giao diện Người dùng . . . 34
5.5 Phân hệ Đặt hàng (Order) . . . 34
5.5.1 Đặc tả Use Case . . . 34
5.5.2 Sơ đồ Hoạt động (Activity Diagram) . . . 36
5.5.3 Sơ đồ Tuần tự (Sequence Diagram) . . . 36
5.5.4 Hợp đồng API (API Contract) . . . 36
5.5.5 Ánh xạ Cơ sở dữ liệu (Database Mapping) . . . 36
5.5.6 Kịch bản Kiểm thử (Test Cases) . . . 38
5.5.7 Giao diện Người dùng . . . 38
5.6 Phân hệ Thanh toán (Payment) . . . 38
5.6.1 Đặc tả Use Case . . . 38
5.6.2 Sơ đồ Hoạt động (Activity Diagram) . . . 41
5.6.3 Sơ đồ Tuần tự (Sequence Diagram) . . . 41
5.6.4 Hợp đồng API (API Contract) . . . 43
5.6.5 Ánh xạ Cơ sở dữ liệu (Database Mapping) . . . 43
5.6.6 Kịch bản Kiểm thử (Test Cases) . . . 43
5.6.7 Giao diện Người dùng . . . 44
5.7 Phân hệ Lịch sử Đơn hàng (Order History) . . . 44
5.7.1 Đặc tả Use Case . . . 44
5.7.2 Sơ đồ Hoạt động (Activity Diagram) . . . 44
5.7.3 Sơ đồ Tuần tự (Sequence Diagram) . . . 44
5.7.4 Hợp đồng API (API Contract) . . . 47
5.7.5 Ánh xạ Cơ sở dữ liệu (Database Mapping) . . . 47
5.7.6 Kịch bản Kiểm thử (Test Cases) . . . 47
5.7.7 Giao diện Người dùng . . . 47

6 HIỆN THỰC CÁC NGHIỆP VỤ NÂNG CAO 49
6.1 Máy trạng thái Đơn hàng (Order State Machine) . . . 49
6.2 Xử lý Thanh toán Bất biến (Idempotent Payment) . . . 51
6.3 Kiểm soát Đồng thời Tồn kho (Concurrency Control) . . . 51
6.4 Nghiệp vụ Khuyến mãi và Điểm tích lũy Thành viên . . . 52
6.4.1 Cơ chế Mã giảm giá (Promotions Engine) . . . 52
6.4.2 Cơ chế Điểm thưởng Thành viên (Loyalty Points) . . . 52

7 HIỆN THỰC HÓA VÀ ĐỐI CHIẾU THIẾT KẾ 54
7.1 Ma trận Ánh xạ Nhiệm vụ GD1 & GD2 sang Mã nguồn . . . 54
7.2 Minh chứng Kỹ thuật Cốt lõi . . . 55
7.2.1 Cơ chế Mã hóa Bcrypt và Cấp phát Token trong AuthService . . . 55
7.2.2 Interactive Transaction và Khóa Lạc quan trong OrdersService . . . 55

7.2.3 Kiểm tra Bất biến Idempotency trong PaymentsService 55

8 ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN 57
8.1 Đánh giá Kết quả Đạt được 57
8.2 Các Hạn chế Hiện tại 57
8.3 Hướng Phát triển trong Tương lai 57

A MA TRẬN TRUY VẾT YÊU CẦU (TRACEABILITY MATRIX) 59

B DANH MỤC API ENDPOINTS 60

C TỪ ĐIỂN DỮ LIỆU (DATA DICTIONARY) 62
C.1 Bảng `users` 62
C.2 Bảng `categories` 62
C.3 Bảng `products` 64
C.4 Bảng `product_size_prices` 64
C.5 Bảng `toppings` 64
C.6 Bảng `promotions` 64
C.7 Bảng `orders` 64
C.8 Bảng `order_items` 64
C.9 Bảng `order_item_toppings` 66
C.10 Bảng `payments` 66
C.11 Bảng `loyalty_transactions` 66
C.12 Bảng `order_status_history` 66

DANH MỤC HÌNH VẼ

1.1 Kế hoạch vận hành Scrum thực tế của Nhóm 3

3.1 Sơ đồ Kiến trúc thành phần tổng thể hệ thống BrewLite 8
3.2 Sơ đồ Triển khai hệ thống đa container với Docker Compose 10
3.3 Sơ đồ Lớp (Class Diagram) tầng Backend NestJS 11

4.1 Sơ đồ Thực thể Liên kết (ERD) toàn hệ thống BrewLite 14

5.1 Sơ đồ Use Case - Phân hệ Xác thực 19
5.2 Sơ đồ Hoạt động - Phân hệ Xác thực 19
5.3 Sơ đồ Tuần tự - Phân hệ Xác thực (Happy path & Exception paths) 21
5.4 Giao diện Đăng nhập và Đăng ký người dùng 22
5.5 Sơ đồ Use Case - Phân hệ Danh mục Sản phẩm 23
5.6 Sơ đồ Hoạt động - Phân hệ Danh mục Sản phẩm 25
5.7 Sơ đồ Tuần tự - Phân hệ Danh mục Sản phẩm 25
5.8 Giao diện Thực đơn sản phẩm BrewLite 26
5.9 Sơ đồ Use Case - Phân hệ Chi tiết Sản phẩm 27
5.10 Sơ đồ Hoạt động - Phân hệ Chi tiết Sản phẩm 29
5.11 Sơ đồ Tuần tự - Phân hệ Chi tiết Sản phẩm 29
5.12 Giao diện Tùy biến và Chi tiết sản phẩm 31
5.13 Sơ đồ Use Case - Phân hệ Giỏ hàng 31
5.14 Sơ đồ Hoạt động - Phân hệ Giỏ hàng 31
5.15 Sơ đồ Tuần tự - Phân hệ Giỏ hàng (Client-side Architecture) 33
5.16 Giao diện Quản lý Giỏ hàng BrewLite 34
5.17 Sơ đồ Use Case - Phân hệ Đặt hàng 36
5.18 Sơ đồ Hoạt động - Phân hệ Đặt hàng 36
5.19 Sơ đồ Tuần tự - Phân hệ Đặt hàng (Interactive Transaction & Pricing Engine) . 37
5.20 Giao diện Xác nhận Đặt hàng và Áp dụng Khuyến mãi 39
5.21 Sơ đồ Use Case - Phân hệ Thanh toán 39
5.22 Sơ đồ Hoạt động - Phân hệ Thanh toán 41
5.23 Sơ đồ Tuần tự - Phân hệ Thanh toán (Idempotent Flow & Gateway Integration) 42
5.24 Giao diện Chọn Phương thức Thanh toán 44
5.25 Sơ đồ Use Case - Phân hệ Lịch sử Đơn hàng 44
5.26 Sơ đồ Hoạt động - Phân hệ Lịch sử Đơn hàng 46
5.27 Sơ đồ Tuần tự - Phân hệ Lịch sử Đơn hàng 46
5.28 Giao diện Lịch sử Đơn hàng của Khách hàng 48

6.1 Sơ đồ Máy trạng thái Đơn hàng BrewLite (9 chuyển tiếp hợp lệ) 50

DANH MỤC BẢNG BIỂU

1.1 Ngăn xếp công nghệ sử dụng trong dự án BrewLite 3

2.1 Bảng tổng hợp Yêu cầu Chức năng (Functional Requirements) 5

5.1 Đặc tả Use Case: Đăng ký và Đăng nhập người dùng 20
5.2 Danh mục API Endpoints - Phân hệ Xác thực 21
5.3 Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Xác thực 22
5.4 Đặc tả Use Case: Xem và Tìm kiếm Danh mục Sản phẩm (UC-CAT-01) 24
5.5 Danh mục API Endpoints - Phân hệ Danh mục Sản phẩm 26
5.6 Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Danh mục 26
5.7 Đặc tả Use Case: Xem chi tiết và Tùy biến món (UC-DET-01) 28
5.8 Danh mục API Endpoints - Phân hệ Chi tiết Sản phẩm 30
5.9 Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Chi tiết Sản phẩm 30
5.10 Đặc tả Use Case: Quản lý Giỏ hàng (UC-CART-01) 32
5.11 Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Giỏ hàng 34
5.12 Đặc tả Use Case: Tạo đơn hàng trực tuyến (UC-ORD-01) 35
5.13 Danh mục API Endpoints - Phân hệ Đặt hàng 37
5.14 Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Đặt hàng 38
5.15 Đặc tả Use Case: Thực hiện Thanh toán Trực tuyến (UC-PAY-01) 40
5.16 Danh mục API Endpoints - Phân hệ Thanh toán 42
5.17 Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Thanh toán 43
5.18 Đặc tả Use Case: Tra cứu Lịch sử và Chi tiết Đơn hàng (UC-HIS-01) 45
5.19 Danh mục API Endpoints - Phân hệ Lịch sử Đơn hàng 47
5.20 Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Lịch sử 47

7.1 Ma trận Đối chiếu Thiết kế GD1 & GD2 sang Mã nguồn thực tế 54

A.1 Ma trận Truy vết Toàn diện Yêu cầu - Thiết kế - Cài đặt - Kiểm thử 59

B.1 Danh mục chi tiết toàn bộ các API Endpoints hệ thống BrewLite 60

Chương 1

GIỚI THIỆU DỰ ÁN

1.1 Tổng quan đề tài và Product Vision

Trong xu hướng chuyển đổi số mạnh mẽ của ngành dịch vụ ăn uống (F&B), mô hình phục vụ truyền thống tại các quầy cà phê thường xuyên đối mặt với tình trạng quá tải trong các khung giờ cao điểm. Khách hàng phải xếp hàng dài chờ đợi, nhân viên thu ngân dễ gặp sai sót khi ghi nhận các tùy chọn đồ uống phức tạp (độ ngọt, lượng đá, kích cỡ, topping), đồng thời việc thanh toán tiền mặt gây chậm trễ trong luồng vận hành.

Hệ thống đặt cà phê trực tuyến **BrewLite** được nghiên cứu và xây dựng nhằm cung cấp giải pháp đặt món tinh gọn (Lite), hiện đại và tiện lợi cho khách hàng. Với tầm nhìn sản phẩm (*Product Vision*):

> *"BrewLite là nền tảng đặt món trực tuyến giúp khách hàng yêu thích cà phê có thể dễ dàng duyệt thực đơn, cá nhân hóa đồ uống theo khẩu vị, đặt hàng nhanh chóng và thanh toán an toàn mà không cần chờ đợi tại quầy; đồng thời giúp cửa hàng tối ưu hóa quy trình xử lý đơn hàng và kiểm soát chặt chẽ tồn kho."*

Mục tiêu cốt lõi của đề tài bao gồm:

- **Trải nghiệm người dùng mượt mà:** Khách hàng có thể tương tác trực quan với thực đơn, phân loại theo danh mục, tùy biến linh hoạt các thông số đồ uống trước khi thêm vào giỏ hàng.
- **Tính toán chi phí minh bạch:** Tích hợp bộ máy tính giá tự động bao gồm phụ thu theo size/topping, áp dụng mã giảm giá (voucher) và đổi điểm thành viên thân thiết (loyalty points).
- **Độ tin cậy và an toàn giao dịch:** Đảm bảo tính toán vẹn dữ liệu trong môi trường đa người dùng thông qua cơ chế kiểm soát đồng thời (Optimistic Locking) cho tồn kho sản phẩm và cơ chế xử lý thanh toán bất biến/trùng lặp (Idempotent Payment).

1.2 Phạm vi hệ thống (MVP Scope)

Trong khuôn khổ đồ án môn học Công nghệ Phần mềm, hệ thống tập trung hoàn thiện các tính năng cốt lõi cho sản phẩm khả dụng tối thiểu (MVP) trong hai giai đoạn phát triển chính (Giai đoạn 1 và Giai đoạn 2):

1. **Phân hệ Xác thực (Auth):** Đăng ký tài khoản, đăng nhập hệ thống, cấp phát mã thông báo JWT an toàn và phân quyền người dùng.
2. **Phân hệ Danh mục sản phẩm (Product Catalog):** Hiển thị danh sách sản phẩm phân nhóm theo danh mục (Cà phê, Trà...), lọc món còn hàng và tìm kiếm.
3. **Phân hệ Chi tiết sản phẩm (Product Detail):** Xem thông tin mô tả chi tiết, hình ảnh, đơn giá và cho phép lựa chọn size, mức đường, mức đá cùng danh sách topping đi kèm.

4. **Phân hệ Giỏ hàng (Cart):** Quản lý các món đã chọn ở phía trình duyệt (Client-side), cập nhật số lượng, chỉnh sửa tùy chọn và tạm tính giá trị đơn hàng.
5. **Phân hệ Đặt hàng (Order):** Tạo đơn hàng chính thức từ giỏ hàng, kiểm tra tính khả dụng của tồn kho, áp dụng mã ưu đãi `WELCOME10`, quy đổi điểm tích lũy và chuyển đổi trạng thái đơn hàng.
6. **Phân hệ Thanh toán (Payment):** Khởi tạo giao dịch thanh toán trực tuyến qua cổng giả lập (Mock Payment Gateway), xử lý bảo mật chống trùng lặp với `Idempotency-Key`.
7. **Phân hệ Lịch sử đơn hàng (Order History):** Tra cứu danh sách các đơn hàng đã thực hiện, xem chi tiết từng đơn hàng và trạng thái thanh toán/xử lý.

Giới hạn phạm vi (Out-of-Scope trong MVP): Để đảm bảo tiến độ và chất lượng cho các nghiệp vụ trọng tâm của khách hàng:

- **Giao diện Pha chế (Barista Web UI):** Mặc dù hệ thống đã hiện thực toàn diện Máy trạng thái đơn hàng (Order State Machine) với đầy đủ 9 chuyển tiếp nghiệp vụ (bao gồm các trạng thái `PREPARING`, `READY`, `COMPLETED`), giao diện web chuyên dụng dành riêng cho nhân viên pha chế tại quầy chưa nằm trong phạm vi giao diện của bản phát hành MVP này.
- **Quản trị viên nâng cao (Advanced Admin Portal):** Hệ thống tập trung vào trải nghiệm đặt hàng của khách hàng (B2C); các chức năng quản trị kho nâng cao và báo cáo doanh thu sẽ được triển khai ở pha tiếp theo.

1.3 Ngăn xếp công nghệ (Technology Stack)

Hệ thống BrewLite được xây dựng trên nền tảng kiến trúc hiện đại, phân tách độc lập giữa Client và Server (Bảng 1.1):

1.4 Quy trình phát triển phần mềm

Dự án áp dụng mô hình phát triển phần mềm linh hoạt **Agile/Scrum**. Quy trình được tổ chức thành các chu kỳ lặp (Sprint) kéo dài 2 tuần:

- **Sprint Planning:** Nhóm họp định kỳ đầu mỗi sprint để phân tích Product Backlog, làm rõ User Story, ước lượng điểm công việc (Story Points) và cam kết phạm vi cho Sprint Backlog.
- **Daily Scrum:** Các thành viên đồng bộ tiến độ hàng ngày, báo cáo các công việc đã hoàn thành, công việc dự kiến và các trở ngại kỹ thuật (blockers).
- **Sprint Review:** Trình diễn các tính năng phần mềm đã hoàn tất đạt tiêu chuẩn chấp nhận (Definition of Done) trước toàn nhóm.
- **Sprint Retrospective:** Đánh giá những điểm làm tốt, các tồn tại về mặt quy trình và kỹ thuật, từ đó đề ra các hành động cải tiến cụ thể cho sprint tiếp theo.

Bảng 1.1: Ngăn xếp công nghệ sử dụng trong dự án BrewLite

Thành phần	Công nghệ	Vai trò và Đặc tính kỹ thuật
Frontend	Next.js 15	Framework React hiện đại sử dụng App Router, hỗ trợ Server Components và Client Components tối ưu SEO và thời gian phản hồi.
	TypeScript	Đảm bảo an toàn kiểu dữ liệu (Type Safety) đồng nhất từ giao diện đến API.
	TailwindCSS	Thiết kế giao diện phản hồi nhanh (Responsive UI) theo chuẩn thiết kế hiện đại.
	Zustand / React Hooks	Quản lý trạng thái giỏ hàng và dữ liệu cục bộ phía Client.
Backend	NestJS 11	Framework Node.js kiến trúc module hướng dịch vụ (Service-Oriented Architecture), hỗ trợ Dependency Injection mạnh mẽ.
	Prisma ORM 6	Ánh xạ quan hệ đối tượng hiện đại, cung cấp Type-safe Database Client và migration tự động.
	Bcrypt	Mã hóa mật khẩu người dùng với cơ chế salt rounds an toàn (10 rounds).
	Passport & JWT	Xác thực và phân quyền bằng cơ chế JSON Web Token không trạng thái (Stateless).
Database	PostgreSQL 16	Hệ quản trị cơ sở dữ liệu quan hệ mạnh mẽ, đảm bảo tính toàn vẹn ACID, hỗ trợ JSONB và lập chỉ mục tối ưu.
Triển khai	Docker & Compose	Đóng gói toàn bộ ứng dụng thành 3 container chuẩn hóa (`frontend`, `backend`, `postgres`), đảm bảo môi trường đồng nhất.

Ghi nhận tiến độ và số liệu thực tế của Nhóm

- **Phân công vai trò: [TODO: Ghi rõ Product Owner (PO): ..., Scrum Master (SM): ...]**
- **Thời gian chu kỳ Sprint:**
  - Sprint 1 (Giai đoạn 1): **[TODO: Từ ngày dd/mm/2026 đến ngày dd/mm/2026]**
  - Sprint 2 (Giai đoạn 2): **[TODO: Từ ngày dd/mm/2026 đến ngày dd/mm/2026]**
- **Số liệu Burndown & Velocity: [TODO: Điền tổng số Story Points cam kết vs hoàn thành trong Sprint 1 và Sprint 2]**
- **Kết quả Retrospective: [TODO: Ghi nhận tóm tắt bài học kinh nghiệm của nhóm sau 2 Sprint]**

Hình 1.1: Kế hoạch vận hành Scrum thực tế của Nhóm

Chương 2

YÊU CẦU HỆ THỐNG

2.1 Các tác nhân hệ thống (Stakeholders & Actors)

Dựa trên phân tích phạm vi nghiệp vụ của hệ thống BrewLite trong Giai đoạn 1 và 2, các tác nhân tương tác trực tiếp với hệ thống được xác định bao gồm:

- **Khách vãng lai (Guest):** Người dùng truy cập hệ thống qua trình duyệt web mà chưa thực hiện đăng nhập. Tác nhân này có thể duyệt danh mục thực đơn, xem chi tiết món ăn, lựa chọn các tùy biến đồ uống và lưu trữ giỏ hàng tạm thời trên trình duyệt. Khi thực hiện thanh toán hoặc đặt hàng, hệ thống yêu cầu xác thực tài khoản.
- **Khách hàng đã đăng ký (Registered Customer):** Khách hàng sở hữu tài khoản định danh trên hệ thống. Tác nhân này có đầy đủ quyền lợi: đặt hàng chính thức, áp dụng mã voucher ưu đãi, tích lũy và đổi điểm thưởng thành viên (Loyalty points), thanh toán trực tuyến và tra cứu toàn bộ lịch sử đơn hàng cá nhân.
- **Cổng thanh toán giả lập (Mock Payment Gateway):** Tác nhân hệ thống bên ngoài mô phỏng trung gian tài chính. Cổng tiếp nhận yêu cầu thanh toán kèm mã định danh bất biến (`Idempotency-Key`), kiểm tra thông tin và trả về kết quả giao dịch (`SUCCESS` hoặc `FAILED`).
- **Hệ thống Quản trị dữ liệu (PostgreSQL Database Engine):** Đóng vai trò thực thi các ràng buộc toàn vẹn, đảm bảo tính bền vững (Durability) và cách ly (Isolation) cho các giao dịch đồng thời.

2.2 Danh mục yêu cầu chức năng (Functional Requirements)

Hệ thống BrewLite được phân rã thành 6 nhóm yêu cầu chức năng chính (FR-01 đến FR-06), tương ứng với các phân hệ nghiệp vụ cốt lõi:

2.3 Danh mục yêu cầu phi chức năng (Non-Functional Requirements)

Các yêu cầu phi chức năng đặt ra các tiêu chuẩn kỹ thuật nhằm đảm bảo chất lượng vận hành của hệ thống:

- **NFR-01 (Hiệu năng - Performance):** Thời gian phản hồi trung bình (API Response Time) của các tác vụ truy vấn danh mục thực đơn và chi tiết sản phẩm không vượt quá 500ms dưới điều kiện tải thông thường.
- **NFR-02 (Bảo mật - Security):**

Bảng 2.1: Bảng tổng hợp Yêu cầu Chức năng (Functional Requirements)

Mã FR	Tên yêu cầu	Mô tả chi tiết
FR-01	Quản lý Xác thực & Tài khoản	Cho phép người dùng đăng ký tài khoản với email duy nhất, họ tên, mật khẩu băm an toàn; Đăng nhập nhận JWT Token hợp lệ; Kiểm soát truy cập bảo vệ các endpoint nghiệp vụ.
FR-02	Duyệt thực đơn & Chi tiết món	Cung cấp danh mục sản phẩm phân nhóm theo phân loại (Cà phê, Trà...); Tìm kiếm món; Xem thông tin chi tiết về kích cỡ, giá phụ thu, mức đường, mức đá và danh sách topping khả dụng.
FR-03	Quản lý Giỏ hàng	Cho phép thêm sản phẩm kèm tùy biến vào giỏ hàng cục bộ; Điều chỉnh số lượng món; Xóa từng món hoặc làm rỗng giỏ; Tạm tính tổng chi phí tự động.
FR-04	Đặt hàng & Xử lý Chiết khấu	Khởi tạo đơn hàng từ giỏ hàng; Kiểm tra tồn kho sản phẩm với Optimistic Locking; Áp dụng mã giảm giá hợp lệ; Quy đổi điểm tích lũy thành viên; Lưu vết snapshot giá tại thời điểm đặt.
FR-05	Thanh toán Trực tuyến	Khởi tạo giao dịch thanh toán; Kiểm tra trùng lặp giao dịch bằng cơ chế `Idempotency-Key` và băm `request_hash`; Cập nhật trạng thái đơn hàng sang `PAID` khi giao dịch thành công.
FR-06	Tra cứu Lịch sử Đơn hàng	Hiển thị danh sách các đơn hàng đã đặt của người dùng theo thứ tự thời gian giảm dần; Xem chi tiết thông tin thanh toán, danh mục món và trạng thái xử lý hiện tại.

  - Toàn bộ mật khẩu người dùng phải được mã hóa một chiều bằng thuật toán `bcrypt` với độ phức tạp 10 salt rounds trước khi lưu trữ vào cơ sở dữ liệu.
  - Hệ thống sử dụng giao thức xác thực JSON Web Token (JWT) không trạng thái (*Stateless*). Tuyệt đối không để lộ trường nhạy cảm `password_hash` trong bất kỳ phản hồi API nào.

- **NFR-03 (Tính toàn vẹn & Tin cậy - Reliability & Idempotency):**
  - Toàn bộ thao tác tạo đơn hàng và trừ kho phải được bọc trong một Interactive Transaction (`$transaction`) của Prisma, đảm bảo nguyên lý ACID.
  - Nghiệp vụ thanh toán phải đảm bảo tính bất biến (*Idempotency*). Các yêu cầu trùng lặp hoặc gửi đồng thời với cùng một `Idempotency-Key` phải được nhận diện và xử lý an toàn, loại trừ hoàn toàn nguy cơ thanh toán lặp (double charging).
- **NFR-04 (Bảo toàn lịch sử giá - Data Immutability):** Giá bán của từng sản phẩm tại thời điểm tạo đơn hàng phải được ghi nhận độc lập vào trường `price_at_order` của bảng chi tiết đơn hàng (`order_items`), đảm bảo lịch sử thanh toán không bị sai lệch khi giá niêm yết trong bảng sản phẩm (`products`) thay đổi trong tương lai.

2.4 User Story tiêu biểu (US-05: Đặt hàng và áp dụng ưu đãi)

Để minh họa mối liên kết giữa yêu cầu của người dùng và thiết kế kỹ thuật, User Story tiêu biểu về quy trình đặt hàng được đặc tả theo chuẩn quốc tế:

User Story US-05: Đặt hàng trực tuyến với mã ưu đãi

Là một: Khách hàng đã đăng nhập vào hệ thống BrewLite,
Tôi muốn: Thực hiện đặt các món trong giỏ hàng và áp dụng mã giảm giá hợp lệ kèm điểm tích lũy thành viên,
Để: Mua được đồ uống yêu thích với chi phí tiết kiệm nhất mà không phải xếp hàng thanh toán tại quầy.

Tiêu chí chấp nhận (Acceptance Criteria - Gherkin Syntax):

Kịch bản 1: *Đặt hàng thành công với mã giảm giá hợp lệ và điểm tích lũy:*

- **Given:** Khách hàng có 3 món đồ uống trong giỏ hàng với tổng giá trị tạm tính là 120.000 VNĐ, tài khoản có 10 điểm thưởng và nhập mã voucher `WELCOME10`.
- **When:** Khách hàng chọn áp dụng mã voucher và nhấn nút "Xác nhận đặt hàng".
- **Then:** Hệ thống áp dụng chiết khấu 10% (12.000 VNĐ), trừ 10 điểm thưởng (tương đương 10.000 VNĐ), tính tổng số tiền thanh toán là 98.000 VNĐ, kiểm tra đủ tồn kho, tạo đơn hàng ở trạng thái `PENDING_PAYMENT` và điều hướng sang trang thanh toán.

Kịch bản 2: *Xử lý ngoại lệ khi số lượng tồn kho không đủ (Out of Stock):*

- **Given:** Khách hàng đặt món Cà phê Sữa Đá với số lượng 2 ly, nhưng tồn kho khả dụng trong cơ sở dữ liệu chỉ còn 1 ly.

- **When:** Khách hàng nhấn ”Xác nhận đặt hàng”.
- **Then:** Hệ thống phát hiện vi phạm số lượng tồn kho trong transaction, hủy bỏ thao tác tạo đơn, trả về mã lỗi `400 Bad Request` kèm thông báo lỗi rõ ràng và giữ nguyên trạng thái giỏ hàng.

Chương 3

KIẾN TRÚC HỆ THỐNG

3.1 Kiến trúc thành phần (Component Architecture)

Hệ thống BrewLite được thiết kế theo mô hình kiến trúc phân tầng dịch vụ (*Layered Service-Oriented Architecture*), đảm bảo tính phân tách trách nhiệm cao (*Separation of Concerns*) và khả năng mở rộng linh hoạt giữa tầng giao diện người dùng, tầng xử lý nghiệp vụ và tầng dữ liệu bền vững.

Sơ đồ kiến trúc thành phần tổng thể của BrewLite được biểu diễn tại Hình 3.1.



Hình 3.1: Sơ đồ Kiến trúc thành phần tổng thể hệ thống BrewLite

Hệ thống bao gồm các thành phần cốt lõi:

1. **Tầng Trình bày (Frontend Presentation Layer - Next.js 15):**
    - Xây dựng bằng Next.js App Router, sử dụng TypeScript và TailwindCSS.
    - Phân chia thành các module giao diện tương ứng với các trang: Xác thực (`/login`, `/register`), Thực đơn (`/products`), Chi tiết món (`/products/[id]`), Giỏ hàng (`/cart`), Đặt hàng (`/checkout`) và Lịch sử (`/orders`).
    - Quản lý trạng thái cục bộ của giỏ hàng thông qua Custom Hook / Zustand và đồng bộ token xác thực lưu trong bộ nhớ an toàn.

2. **Tầng Xử lý Nghiệp vụ (Backend Application Layer - NestJS 11):** Tổ chức theo các Module chuyên biệt, tận dụng cơ chế Dependency Injection (DI):
   - **AuthModule:** Đảm nhận đăng ký, băm mật khẩu bằng `bcrypt`, đăng nhập và cấp phát JSON Web Token; bảo vệ hệ thống thông qua `JwtAuthGuard`.
   - **ProductsModule:** Quản lý danh mục (`categories`), sản phẩm (`products`), các tùy chọn size/đường/đá/topping và kiểm tra tính khả dụng của kho hàng.
   - **OrdersModule:** Trung tâm điều phối quy trình đặt hàng, tích hợp chặt chẽ với máy trạng thái đơn hàng (`OrderStateMachine`) và kiểm soát giao dịch tạo đơn.
   - **PricingModule, PromotionsModule & LoyaltyModule:** Các động cơ tính toán độc lập thực thi chính sách phụ thu tùy biến, kiểm tra hợp lệ của mã giảm giá `WELCOME10` và quy đổi điểm tích lũy thành viên.
   - **PaymentsModule:** Xử lý thanh toán trực tuyến, quản lý bảng idempotency (`PaymentIdempoter`) và kết nối giao tiếp với cổng thanh toán.
   - **Common / Core Services:** Bộ lọc ngoại lệ toàn cục (`AllExceptionsFilter`) và bộ chuẩn hóa phản hồi (`TransformInterceptor`) đảm bảo mọi giao tiếp API luôn tuân thủ cấu trúc đồng nhất `{success, data, message, timestamp}`.
3. **Tầng Cơ sở dữ liệu (Data Access Layer):** Sử dụng **Prisma ORM 6** cung cấp Type-safe Client kết nối trực tiếp đến hệ quản trị cơ sở dữ liệu quan hệ **PostgreSQL 16**.
4. **Hệ thống ngoại vi (External Service): Mock Payment Gateway** giả lập cổng thanh toán trực tuyến của bên thứ ba, cung cấp endpoint xử lý thẻ/ví điện tử và phản hồi kết quả giao dịch về backend.

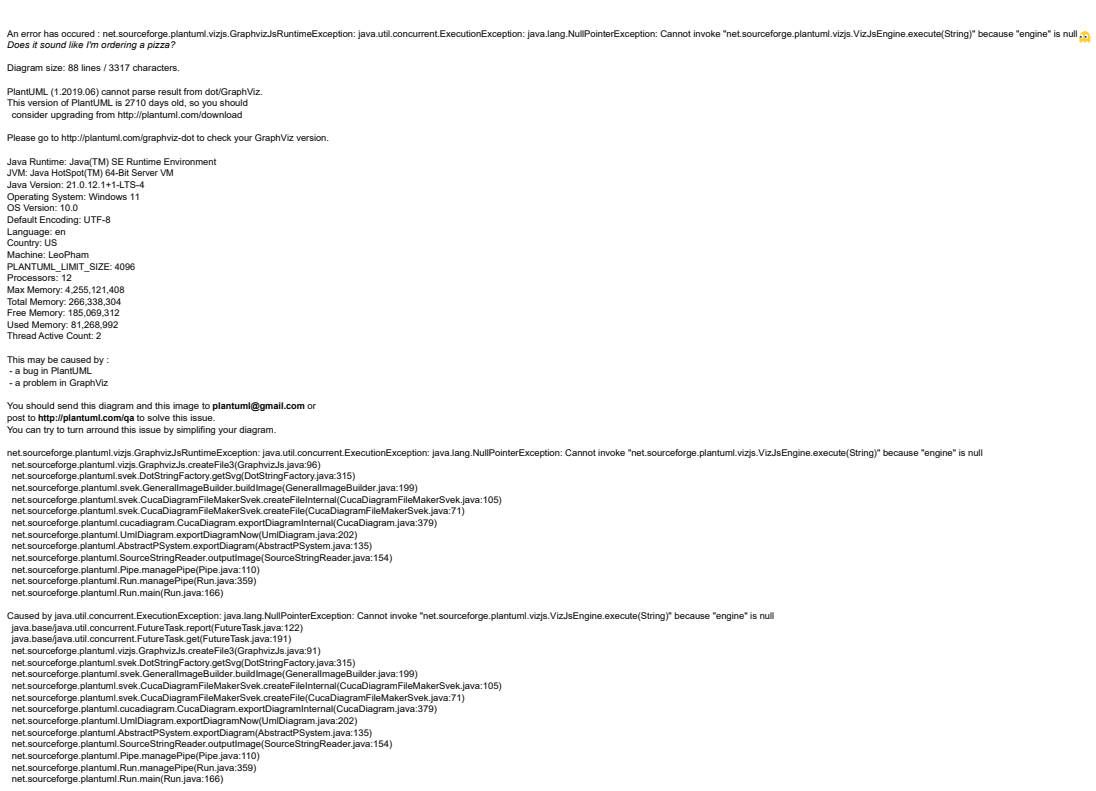
3.2 Kiến trúc triển khai với Docker (Deployment Architecture)

Toàn bộ môi trường ứng dụng BrewLite được đóng gói và cấu hình tự động thông qua **Docker** và **Docker Compose**, giúp loại bỏ sự sai lệch giữa môi trường phát triển (Development) và môi trường kiểm thử/triển khai.

Sơ đồ kiến trúc triển khai container được minh họa tại Hình 3.2.

Hệ thống bao gồm 3 container độc lập kết nối thông qua mạng nội bộ Docker bridge (`brewlite-network`

- **Frontend Container (`brewlite-frontend`):**
  - Base image: `node:20-alpine`.
  - Port ánh xạ: `3000:3000`.
  - Biến môi trường: `NEXT_PUBLIC_API_BASE_URL=http://localhost:3001/api/v1`.
- **Backend Container (`brewlite-backend`):**
  - Base image: `node:20-alpine`.
  - Port ánh xạ: `3001:3001`.
  - Biến môi trường: `DATABASE_URL=postgresql://user:pass@postgres:5432/brewlite`, `JWT_SECRET=...`, `JWT_EXPIRES_IN=24h`, `PORT=3001`.



Hình 3.2: Sơ đồ Triển khai hệ thống đa container với Docker Compose

  - Phụ thuộc (`depends_on`): Chờ container `postgres` sẵn sàng nhận kết nối trước khi khởi chạy và tự động thực thi lệnh `prisma migrate deploy`.

- **Database Container (`brewlite-postgres`):**
  - Base image: `postgres:16-alpine`.
  - Port ánh xạ: `5432:5432`.
  - Lưu trữ bền vững (`Persistent Storage`): Mount volume Docker `postgres_data` vào thư mục `/var/lib/postgresql/data`, đảm bảo dữ liệu không bị thất thoát khi container khởi động lại hoặc dừng hoạt động.

3.3 Sơ đồ lớp Backend (Backend Class Diagram)

Cấu trúc hướng đối tượng của hệ thống Backend được thể hiện thông qua Sơ đồ lớp tại Hình 3.3. Sơ đồ mô tả rõ nét mối liên kết giữa các tầng Controller, Service, DTO và các Entity của Prisma Client.

Các lớp dịch vụ chính bao gồm:

- `AuthService`: Xử lý logic mã hóa mật khẩu, kiểm tra trùng lặp email và sinh mã xác thực JWT.
- `ProductsService`: Chịu trách nhiệm truy vấn danh mục, lọc món và thực thi lệnh cập nhật phiên bản tồn kho (`version`) khi có giao dịch.
- `OrdersService`: Lớp điều phối chính của quy trình mua hàng. Quản lý việc tạo đơn trong transaction, phối hợp cùng `PricingService`, `PromotionsService` và `LoyaltyService` để xác định số tiền chính xác, sau đó kiểm tra và trừ tồn kho.



Hình 3.3: Sơ đồ Lớp (Class Diagram) tầng Backend NestJS

- `PaymentsService`: Khởi tạo yêu cầu thanh toán, quản lý và kiểm tra bảng `PaymentIdempotency`, gọi Mock Gateway và cập nhật trạng thái đơn hàng.

3.4 Luồng tương tác nghiệp vụ tổng thể (Overall Business Flow)

Luồng tương tác nghiệp vụ từ đầu đến cuối (End-to-End) của một khách hàng trên hệ thống diễn ra tuần tự qua các giai đoạn:

1. **Giai đoạn 1 - Khám phá và Chọn món:** Khách hàng duyệt danh mục thực đơn tại trang chủ (`/products`), chọn đồ uống để xem chi tiết (`/products/[id]`), cấu hình kích cỡ (Size M/L), mức đường, mức đá và các món topping đi kèm. Sau đó món ăn được đưa vào giỏ hàng cục bộ (`Cart`).

2. **Giai đoạn 2 - Xác thực tài khoản:** Khi người dùng chọn tiến hành đặt hàng (`Checkout`), nếu chưa đăng nhập, hệ thống sẽ tự động điều hướng sang trang Đăng nhập/Đăng ký để xác thực danh tính.

3. **Giai đoạn 3 - Đặt hàng và Tính giá:** Khách hàng nhập địa chỉ giao hàng hoặc chọn nhận tại quầy, áp dụng mã ưu đãi `WELCOME10` và chọn số điểm tích lũy muốn đổi. Backend thực thi kiểm tra tồn kho đồng thời qua cơ chế Optimistic Lock, tạo bản ghi đơn hàng ở trạng thái `PENDING_PAYMENT`.

4. **Giai đoạn 4 - Thanh toán Trực tuyến:** Hệ thống chuyển hướng khách hàng sang màn hình thanh toán (`/payment`). Khách hàng chọn phương thức (Thẻ ngân hàng hoặc Ví điện tử). Lệnh thanh toán được gửi kèm `Idempotency-Key`. Mock Gateway xác nhận giao dịch thành công.

5. **Giai đoạn 5 - Cập nhật và Hoàn tất:** Đơn hàng được chuyển trạng thái sang `PAID`, ghi nhận điểm thưởng mới vào tài khoản khách hàng, giải phóng giỏ hàng và lưu vết vào Lịch sử đơn hàng (`/orders`).

Chương 4

THIẾT KẾ CƠ SỞ DỮ LIỆU

4.1 Sơ đồ Thực thể Liên kết (ERD) toàn hệ thống

Cơ sở dữ liệu của hệ thống BrewLite được chuẩn hóa theo dạng chuẩn 3NF, thiết kế bám sát tập tin mô hình dữ liệu `backend/prisma/schema.prisma`. Toàn bộ hệ thống bao gồm 12 thực thể quan hệ, đảm bảo tính toàn vẹn dữ liệu, hiệu năng truy vấn và hỗ trợ chuyên sâu cho các nghiệp vụ phức tạp như kiểm soát đồng thời (Optimistic Locking) và thanh toán bất biến (Idempotency).

Sơ đồ ERD toàn hệ thống được trình bày ở chế độ khổ ngang (Landscape) tại Hình 4.1.

An error has occured : net.sourceforge.plantuml.vizjs.GraphvizJsRuntimeException: java.util.concurrent.ExecutionException: java.lang.NullPointerException: Cannot invoke "net.sourceforge.plantuml.vizjs.VizJsEngine.execute(String)" because "engine" is null
By Grabthar's hammer, by the sons of Worvan, you shall be avenged.

Diagram size: 202 lines / 5161 characters.

PlantUML (1.2019.06) cannot parse result from dot/GraphViz.
This version of PlantUML is 2710 days old, so you should
consider upgrading from http://plantuml.com/download

Please go to http://plantuml.com/graphviz-dot to check your GraphViz version.

Java Runtime: Java(TM) SE Runtime Environment
JVM: Java HotSpot(TM) 64-Bit Server VM
Java Version: 21.0.12.1+1-LTS-4
Operating System: Windows 11
OS Version: 10.0
Default Encoding: UTF-8
Language: en
Country: US
Machine: LeoPham
PLANTUML_LIMIT_SIZE: 4096
Processors: 12
Max Memory: 4,255,121,408
Total Memory: 266,338,304
Free Memory: 147,052,280
Used Memory: 119,286,024
Thread Active Count: 2

This may be caused by :
- a bug in PlantUML
- a problem in GraphViz

You should send this diagram and this image to **plantuml@gmail.com** or
post to **http://plantuml.com/qa** to solve this issue.
You can try to turn arround this issue by simplifing your diagram.

net.sourceforge.plantuml.vizjs.GraphvizJsRuntimeException: java.util.concurrent.ExecutionException: java.lang.NullPointerException: Cannot invoke "net.sourceforge.plantuml.vizjs.VizJsEngine.execute(String)" because "engine" is null
net.sourceforge.plantuml.vizjs.GraphvizJs.createFile3(GraphvizJs.java:96)
net.sourceforge.plantuml.svek.DotStringFactory.getSvg(DotStringFactory.java:315)
net.sourceforge.plantuml.svek.GeneralImageBuilder.buildImage(GeneralImageBuilder.java:199)
net.sourceforge.plantuml.svek.CucaDiagramFileMakerSvek.createFileInternal(CucaDiagramFileMakerSvek.java:105)
net.sourceforge.plantuml.svek.CucaDiagramFileMakerSvek.createFile(CucaDiagramFileMakerSvek.java:71)
net.sourceforge.plantuml.cucadiagram.CucaDiagram.exportDiagramInternal(CucaDiagram.java:379)
net.sourceforge.plantuml.classdiagram.ClassDiagram.exportDiagramInternal(ClassDiagram.java:194)
net.sourceforge.plantuml.UmlDiagram.exportDiagramNow(UmlDiagram.java:202)
net.sourceforge.plantuml.AbstractPSystem.exportDiagram(AbstractPSystem.java:135)
net.sourceforge.plantuml.SourceStringReader.outputImage(SourceStringReader.java:154)
net.sourceforge.plantuml.Pipe.managePipe(Pipe.java:110)
net.sourceforge.plantuml.Run.managePipe(Run.java:359)
net.sourceforge.plantuml.Run.main(Run.java:166)

Caused by java.util.concurrent.ExecutionException: java.lang.NullPointerException: Cannot invoke "net.sourceforge.plantuml.vizjs.VizJsEngine.execute(String)" because "engine" is null
java.base/java.util.concurrent.FutureTask.report(FutureTask.java:122)
java.base/java.util.concurrent.FutureTask.get(FutureTask.java:191)
net.sourceforge.plantuml.vizjs.GraphvizJs.createFile3(GraphvizJs.java:91)
net.sourceforge.plantuml.svek.DotStringFactory.getSvg(DotStringFactory.java:315)
net.sourceforge.plantuml.svek.GeneralImageBuilder.buildImage(GeneralImageBuilder.java:199)
net.sourceforge.plantuml.svek.CucaDiagramFileMakerSvek.createFileInternal(CucaDiagramFileMakerSvek.java:105)
net.sourceforge.plantuml.svek.CucaDiagramFileMakerSvek.createFile(CucaDiagramFileMakerSvek.java:71)
net.sourceforge.plantuml.cucadiagram.CucaDiagram.exportDiagramInternal(CucaDiagram.java:379)
net.sourceforge.plantuml.classdiagram.ClassDiagram.exportDiagramInternal(ClassDiagram.java:194)
net.sourceforge.plantuml.UmlDiagram.exportDiagramNow(UmlDiagram.java:202)
net.sourceforge.plantuml.AbstractPSystem.exportDiagram(AbstractPSystem.java:135)
net.sourceforge.plantuml.SourceStringReader.outputImage(SourceStringReader.java:154)
net.sourceforge.plantuml.Pipe.managePipe(Pipe.java:110)
net.sourceforge.plantuml.Run.managePipe(Run.java:359)
net.sourceforge.plantuml.Run.main(Run.java:166)

Hình 4.1: Sơ đồ Thực thể Liên kết (ERD) toàn hệ thống BrewLite

4.2 Chi tiết 12 thực thể trong Cơ sở dữ liệu

Mô hình dữ liệu bao gồm 12 bảng được phân chia theo các nhóm nghiệp vụ chính:

1. **Nhóm Người dùng và Định danh:**
    - **users:** Lưu trữ thông tin định danh khách hàng. Gồm các trường: `id` (PK, cuid), `email` (Unique, chuỗi định danh duy nhất), `password_hash` (mật khẩu băm Bcrypt 10 rounds), `full_name`, `loyalty_points` (điểm tích lũy hiện có, mặc định 0), `created_at`, `updated_at`.
2. **Nhóm Danh mục và Sản phẩm:**
    - **categories:** Phân loại thực đơn (Cà phê, Trà...). Gồm: `id` (PK), `name` (Unique), `slug` (Unique), `display_order` (thứ tự hiển thị trên tab giao diện), `created_at`, `updated_at`.
    - **products:** Thông tin đồ uống. Gồm: `id` (PK), `category_id` (FK $\rightarrow$ `categories.id`), `name`, `base_price` (Decimal(12,0) tiền VNĐ nguyên chẵn), `image_url`, `stock` (số lượng tồn kho khả dụng), `version` (Int, default 0 - trường phiên bản phục vụ Optimistic Locking), `is_available` (Boolean), `created_at`, `updated_at`.
    - **product_size**$_p rices : Cuhnhphthutheotngkchc(S, M, L)chotngsnphm.Gm :$ **id**$(PK),$**product_id**$(FK \rightarrow$ **products.id, Cascade), size (Enum ProductSize: S, M, L), price_adjustment (Decimal(12,0)). Có ràng buộc duy nhất @@unique([product_id, size]).**
    - **toppings: Danh mục topping dùng kèm. Gồm: id (PK), name (Unique), price (Decimal(12,0)), stock (Int, default 100), is_available (Boolean), created_at, updated_at.**
3. **Nhóm Khuyến mãi (Promotions):**
    - **promotions:** Quản lý mã giảm giá voucher. Gồm: `id` (PK), `code` (Unique, ví dụ: `WELCOME10`, `GIAM15K`), `description`, `discount_percent` (tỷ lệ giảm theo %), `discount_amount` (tiền giảm cố định), `max_discount` (mức giảm trần tối đa, ví dụ 50.000 VNĐ), `min_order_amount` (giá trị đơn tối thiểu, ví dụ 100.000 VNĐ), `start_date`, `end_date`, `is_active` (Boolean).
4. **Nhóm Đơn hàng và Chi tiết:**
    - **orders:** Bảng trung tâm lưu trữ thông tin đơn hàng. Gồm: `id` (PK), `order_number` (Int, Unique, autoincrement - mã hiển thị cho khách dạng #1042), `user_id` (FK $\rightarrow$ `users.id`), `promotion_id` (FK $\rightarrow$ `promotions.id`, có thể null), `status` (Enum `OrderStatus`), `total_amount` (tạm tính trước giảm), `discount_amount` (số tiền được giảm), `final_amount` (số tiền thực trả), `points_earned` (số điểm thưởng tích được từ đơn), `paid_at` (thời điểm thanh toán), `version` (Int, default 0 - Optimistic Locking khi chuyển trạng thái), `created_at`, `updated_at`.
    - **order_items:** Chi tiết từng món trong đơn. Gồm: `id` (PK), `order_id` (FK $\rightarrow$ `orders.id`, Cascade), `product_id` (FK $\rightarrow$ `products.id`), `size` (Enum `ProductSize`), `quantity` (Int), `unit_price` (Decimal(12,0) - đơn giá snapshot đã cộng phụ thu size), `line_total` (Decimal(12,0) = (unitPrice + toppings) $\times$ quantity).

- **order_item_toppings:** Lưu trữ topping kèm theo từng món. Gồm: id (PK), order_item_id (FK → order_items.id, Cascade), topping_id (FK → toppings.id), price (Decimal(12,0) - snapshot giá topping tại thời điểm đặt).
- **order_status**$_h istory : Luvtlchschuynitrngthinhng(phcvgimst9chuyntipcamytrngthi).$**id**$(PK),$**order_id**$(FK \rightarrow$ **orders.id, Cascade), from_status (OrderStatus?), to_status (OrderStatus), changed_by (String?), reason (String?), created_at.**

5. **Nhóm Thanh toán và Điểm thành viên:**

   - **payments:** Lưu thông tin các lần thanh toán. Gồm: id (PK), order_id (FK → orders.id, quan hệ 1-N hỗ trợ retry khi thanh toán thất bại), idempotency_key (Unique, chuỗi UUID chống trùng lặp), request_hash (mã băm SHA-256 nội dung request), amount (Decimal(12,0)), method (Enum: BANK_CARD, E_WALLET), status (Enum: PENDING, SUCCESS, FAILED), failure_reason, transaction_ref, created_at.
   - **loyalty_transactions:** Nhật ký biến động điểm thưởng. Gồm: id (PK), user_id (FK → users.id), order_id (FK → orders.id?), points (Int), type (Enum: EARNED, SPENT, REFUNDED), reason, created_at.

4.3 Ràng buộc toàn vẹn và Chiến lược đánh chỉ mục

- **Ràng buộc Khóa ngoại (Referential Integrity):** Sử dụng cơ chế onDelete: Cascade cho các quan hệ cha-con gắn kết chặt chẽ: khi xóa đơn hàng (Order), hệ thống tự động dọn dẹp các dòng tương ứng trong order_items, order_item_toppings, order_status_history và payments. Ngược lại, quan hệ giữa Product và OrderItem áp dụng cơ chế bảo vệ (Restrict) để ngăn ngừa việc xóa mất dấu sản phẩm đã có trong hóa đơn.
- **Ràng buộc Duy nhất (Unique Constraints):** Thiết lập chỉ mục duy nhất trên users(email), categories(slug), categories(name), promotions(code), orders(order_number), payments(idempotency_key) và bộ đôi product_size_prices(product_id, size).
- **Chỉ mục Tối ưu truy vấn (Query Optimization Indexes):** Đánh chỉ mục B-tree trên các trường khóa ngoại và các trường thường xuyên xuất hiện trong mệnh đề lọc/sắp xếp:
  - orders(user_id), orders(status): tăng tốc độ truy vấn danh sách lịch sử đơn hàng.
  - order_items(order_id): tăng tốc độ nạp dữ liệu hóa đơn chi tiết.
  - payments(order_id): hỗ trợ tra cứu các giao dịch theo đơn hàng.
  - loyalty_transactions(user_id): tối ưu hóa việc tính toán lịch sử điểm thưởng.

4.4 Thiết kế CSDL hỗ trợ nghiệp vụ nâng cao

Cơ sở dữ liệu của BrewLite được tính toán kỹ lưỡng ngay từ thiết kế schema để đáp ứng các bài toán kỹ thuật phức tạp:

1. **Kiểm soát Đồng thời (Optimistic Locking):** Trường `version` trên bảng `products` và `orders` cho phép phát hiện xung đột dữ liệu mà không cần khóa bi quan (Pessimistic lock) gây nghẽn cổ chai. Câu lệnh cập nhật kho luôn đi kèm điều kiện `WHERE id = :id AND version = :currentVersion`, tự động tăng `version = version + 1`.
2. **Thanh toán Bất biến (Idempotent Payment):** Trường `idempotency_key` có tính duy nhất toàn cục kết hợp cùng `request_hash` trong bảng `payments` giúp hệ thống ngay lập tức nhận diện các request gửi lại từ client hoặc lỗi mạng, trả về kết quả đã xử lý mà không thực hiện gọi lại cổng thanh toán.
3. **Snapshot Lịch sử Giá (Price Immutability):** Các trường `unit_price` trong `order_items` và `price` trong `order_item_toppings` sao lưu chính xác giá trị tại thời điểm khách hàng đặt, đảm bảo giá trị hóa đơn luôn cố định bất biến theo thời gian.

Chương 5

THIẾT KẾ CHI TIẾT CÁC PHÂN HỆ CHỨC NĂNG

Chương này trình bày chi tiết thiết kế kỹ thuật cho toàn bộ 7 phân hệ chức năng cốt lõi của hệ thống BrewLite. Mỗi phân hệ được chuẩn hóa theo cấu trúc toàn diện bao gồm:

1. **Đặc tả Use Case:** Sơ đồ Use Case và bảng đặc tả nghiệp vụ chi tiết (Tiền điều kiện, Hậu điều kiện, Luồng chính, Luồng ngoại lệ).
2. **Sơ đồ Hoạt động (Activity Diagram):** Mô tả luồng xử lý quyết định nghiệp vụ.
3. **Sơ đồ Tuần tự (Sequence Diagram):** Ánh xạ tương tác chuẩn xác từ Người dùng → Frontend Component → Controller → Service → Prisma/DB.
4. **Hợp đồng API (API Contract):** Đặc tả Endpoint, phương thức HTTP, DTO validation và cấu trúc phản hồi.
5. **Ánh xạ Cơ sở dữ liệu (Database Mapping):** Thao tác bảng, trường dữ liệu và ràng buộc ACID.
6. **Kịch bản Kiểm thử (Test Cases):** Bảng các ca kiểm thử nghiệp vụ và giao diện.
7. **Giao diện Người dùng (User Interface):** Vị trí lưu giữ minh chứng cài đặt giao diện.

5.1 Phân hệ Xác thực Người dùng (Auth)

5.1.1 Đặc tả Use Case

Phân hệ Xác thực quản lý vòng đời tài khoản của người dùng, cung cấp cơ chế đăng ký và đăng nhập bảo mật bằng JSON Web Token (JWT). Sơ đồ Use Case được minh họa tại Hình 5.1.

5.1.2 Sơ đồ Hoạt động (Activity Diagram)

Sơ đồ Hoạt động mô tả chi tiết quy trình xử lý luồng đăng nhập và kiểm tra xác thực người dùng được biểu diễn tại Hình 5.2.

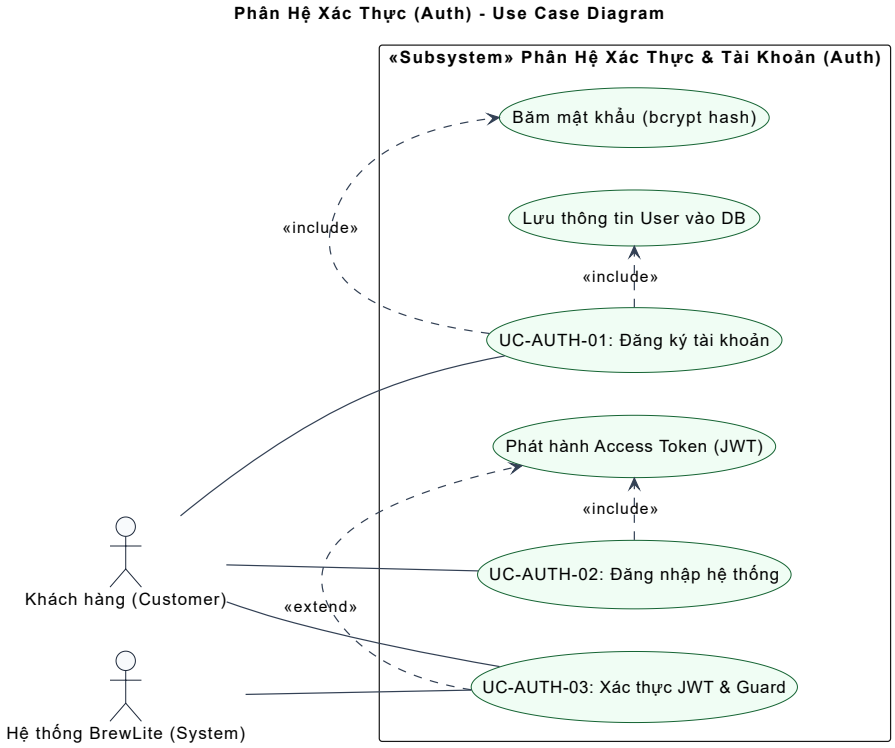
5.1.3 Sơ đồ Tuần tự (Sequence Diagram)

Sơ đồ Tuần tự phản ánh chuẩn xác các lời gọi hàm từ giao diện qua Controller đến Service và Prisma DB tại Hình 5.3.

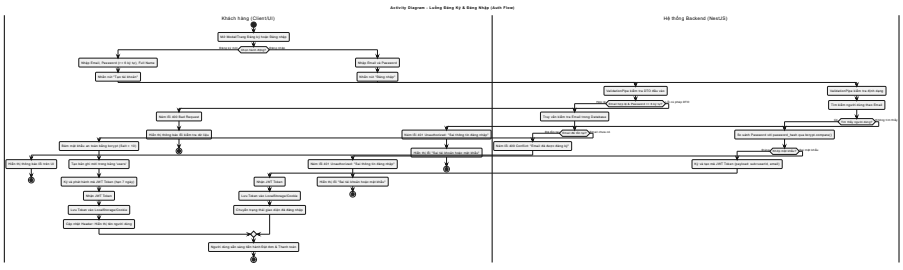
5.1.4 Hợp đồng API (API Contract)

5.1.5 Ánh xạ Cơ sở dữ liệu (Database Mapping)

Mọi thao tác của AuthModule tác động trực tiếp lên bảng `users`:



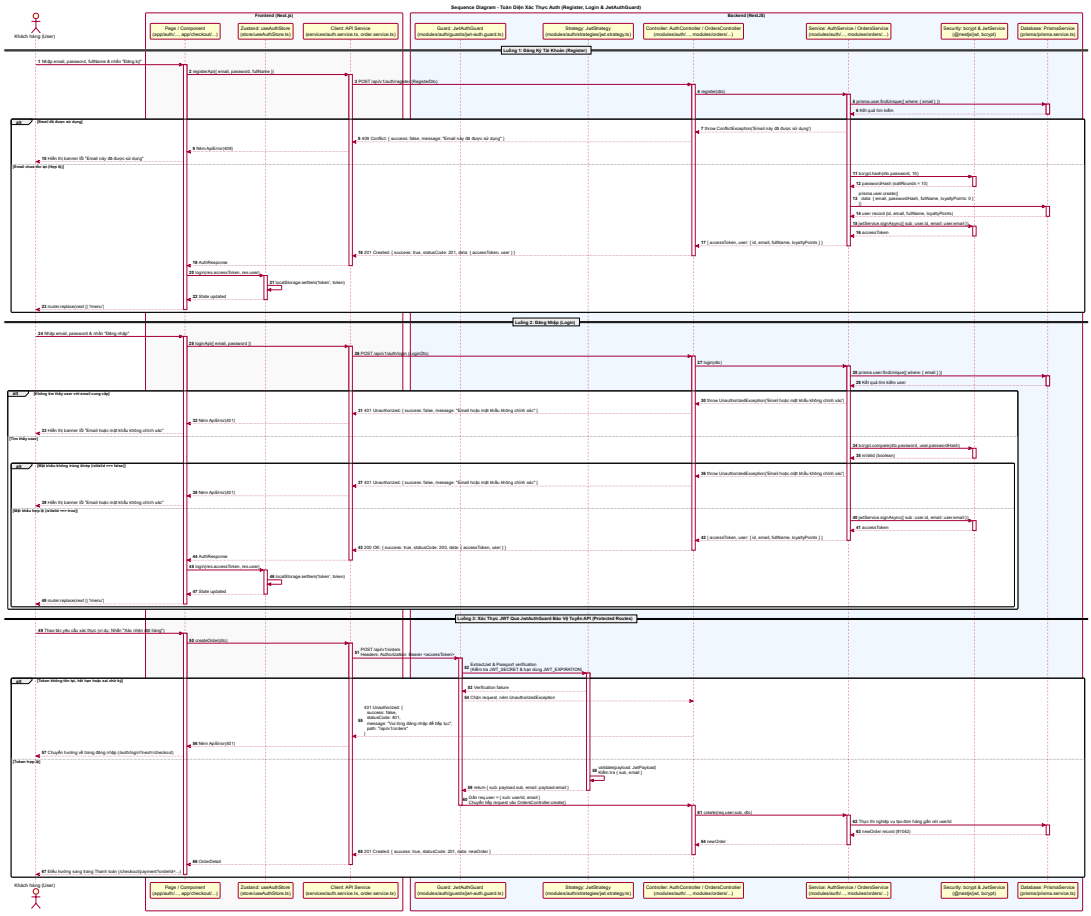
Hình 5.1: Sơ đồ Use Case - Phân hệ Xác thực



Hình 5.2: Sơ đồ Hoạt động - Phân hệ Xác thực

Bảng 5.1: Đặc tả Use Case: Đăng ký và Đăng nhập người dùng

Mục	**Đăng ký (UC-AUTH-01)**	**Đăng nhập (UC-AUTH-02)**
Actor	Khách vãng lai (Guest)	Khách hàng đã có tài khoản
Tiền điều kiện	Chưa đăng nhập; địa chỉ email chưa tồn tại trên hệ thống.	Đã có tài khoản trên hệ thống; tài khoản ở trạng thái hoạt động.
Hậu điều kiện	Bản ghi mới được tạo trong bảng `users`; mật khẩu được băm Bcrypt 10 rounds; khởi tạo 0 điểm loyalty.	Cấp phát mã thông báo JWT `accessTok` lưu trữ tại client để xác thực các re-quest kế tiếp.
Luồng chính	1. Người dùng nhập email, mật khẩu ($\geq$ 6 ký tự), họ tên. 2. Nhấn ”Đăng ký”. 3. Hệ thống kiểm tra hợp lệ DTO và tính duy nhất của email. 4. Băm mật khẩu bằng `bcrypt.hash(password, 10)`. 5. Tạo người dùng trong DB và trả về thông tin (loại trừ `password_hash`).	1. Người dùng nhập email và mật khẩu. 2. Nhấn ”Đăng nhập”. 3. Hệ thống tìm người dùng theo email.



Hình 5.3: Sơ đồ Tuần tự - Phân hệ Xác thực (Happy path & Exception paths)

Bảng 5.2: Danh mục API Endpoints - Phân hệ Xác thực

Phương thức	**Endpoint**	**Mô tả Request Body**	**Phản hồi chính**
`POST`	`/api/v1/auth/register`	`{email, password, fullName}` (Email hợp lệ, password $\geq$ 6 ký tự)	`201 Created`: Trả về thông tin người dùng mới (loại trừ passwordHash). Lỗi: `400`, `409`.
`POST`	`/api/v1/auth/login`	`{email, password}`	`200 OK`: Trả về `accessToken` và thông tin người dùng. Lỗi: `400`, `401`.

- **Đăng ký:** Gọi `prisma.user.findUnique({ where: { email } })`. Nếu tồn tại → throw `ConflictException`. Nếu chưa, thực thi `bcrypt.hash(password, 10)` và gọi `prisma.user.create({ data: { email, passwordHash, fullName, loyaltyPoints: 0 } })`.
- **Đăng nhập:** Gọi `prisma.user.findUnique({ where: { email } })`. Nếu không tìm thấy hoặc `bcrypt.compare()` trả về false → throw `UnauthorizedException`.
- **Bảo mật dữ liệu:** Trường `passwordHash` tuyệt đối được lược bỏ trước khi gán vào đối tượng trả về thông qua destructuring: `const { passwordHash, ...userWithoutPassword } = user`.

5.1.6 Kịch bản Kiểm thử (Test Cases)

Bảng 5.3: Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Xác thực

Mã TC	Loại	Tiền điều kiện	Endpoint / Màn hình	Kết quả mong đợi	T
`TC-AUTH-API-01`	Hợp lệ	Email chưa tồn tại	`POST /auth/register`	HTTP 201, tạo user mới, băm bcrypt 10 rounds	C
`TC-AUTH-API-03`	Lỗi	Email đã tồn tại	`POST /auth/register`	HTTP 409, message ”Email đã được sử dụng”	C
`TC-AUTH-API-05`	Hợp lệ	Đã có tài khoản	`POST /auth/login`	HTTP 200, cấp JWT accessToken	C
`TC-AUTH-API-07`	Lỗi	Mật khẩu không đúng	`POST /auth/login`	HTTP 401, message ”Email hoặc mật khẩu không chính xác”	C
`TC-AUTH-UI-01`	Giao diện	Đang ở trang `/login`	Màn hình Đăng nhập	Hiển thị form, nút ”Đăng nhập”, chuyển hướng thành công	C

5.1.7 Giao diện Người dùng

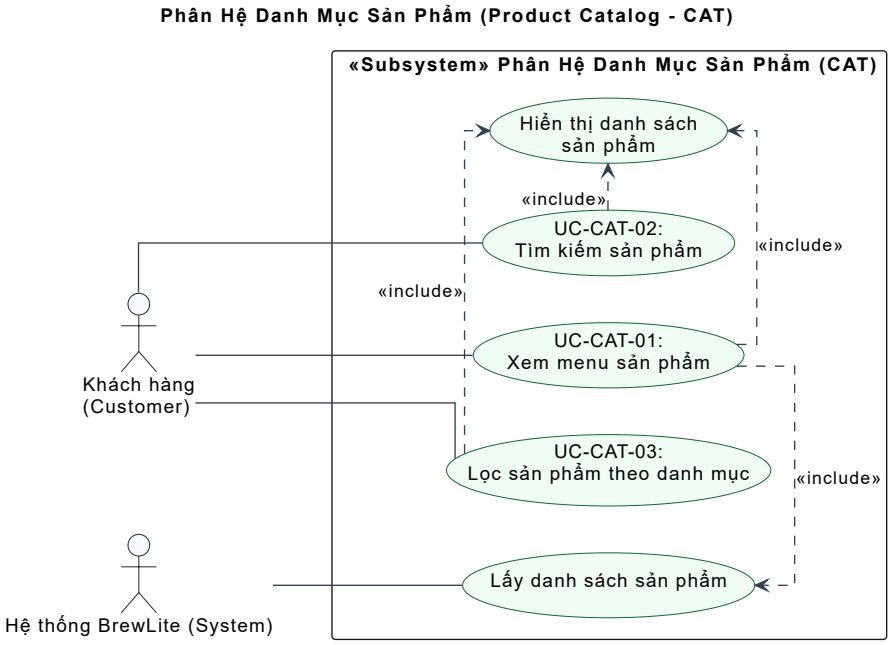
[TODO: Chèn ảnh chụp màn hình Giao diện Đăng nhập (`/login`) và Đăng ký (`/register`)]

Hình 5.4: Giao diện Đăng nhập và Đăng ký người dùng

5.2 Phân hệ Danh mục Sản phẩm (Product Catalog)

5.2.1 Đặc tả Use Case

Phân hệ Danh mục Sản phẩm cho phép người dùng khám phá toàn bộ thực đơn của BrewLite, tìm kiếm đồ uống theo tên và lọc theo các phân loại như Cà phê hoặc Trà. Sơ đồ Use Case được biểu diễn tại Hình 5.5.



Hình 5.5: Sơ đồ Use Case - Phân hệ Danh mục Sản phẩm

5.2.2 Sơ đồ Hoạt động (Activity Diagram)

Quy trình nạp và lọc danh mục sản phẩm được mô tả tại Hình 5.6.

5.2.3 Sơ đồ Tuần tự (Sequence Diagram)

Sơ đồ Tuần tự phản ánh quá trình truy vấn danh mục và sản phẩm từ Frontend đến Backend Controller, Service và Prisma tại Hình 5.7.

5.2.4 Hợp đồng API (API Contract)

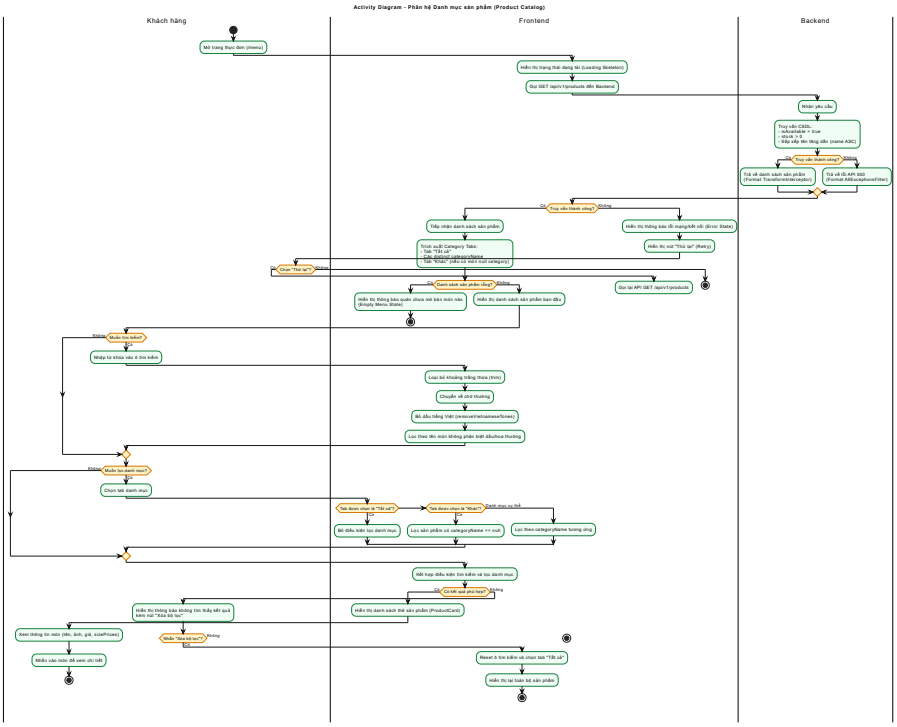
5.2.5 Ánh xạ Cơ sở dữ liệu (Database Mapping)

- Truy vấn danh mục: `prisma.category.findMany({ orderBy: { displayOrder: 'asc' } })`.
- Truy vấn sản phẩm: `prisma.product.findMany({ where: { ...filter }, include: { sizePrices: true, category: true } })`.

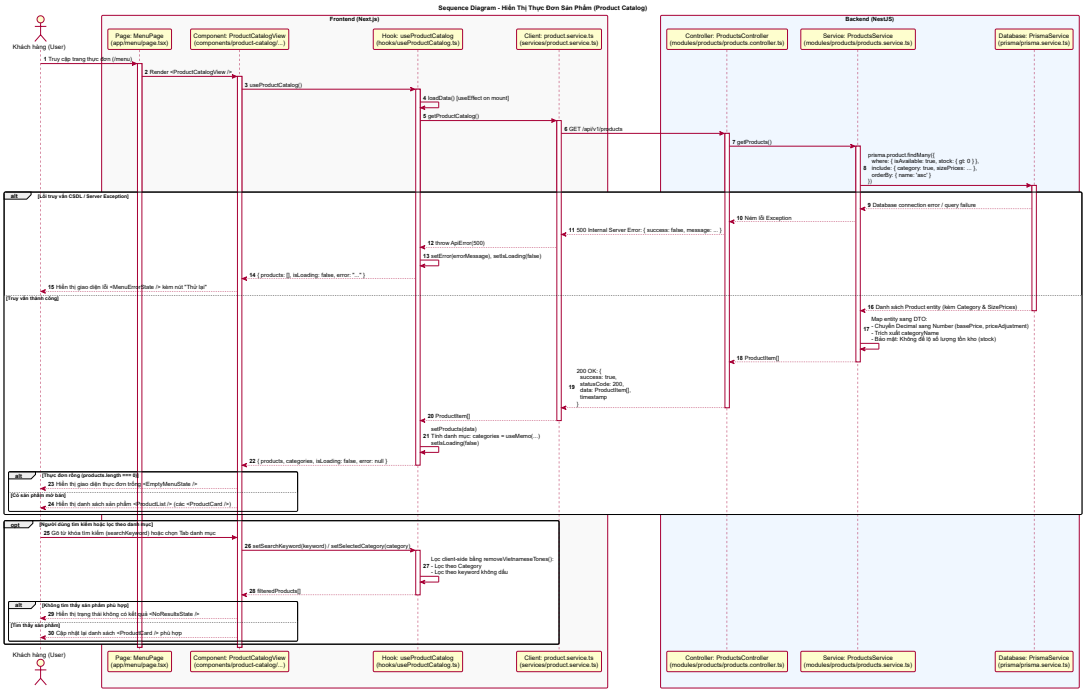
Bảng 5.4: Đặc tả Use Case: Xem và Tìm kiếm Danh mục Sản phẩm (UC-CAT-01)

Mục	
Actor	
Tiền điều kiện	
Hậu điều kiện	
Luồng chính	

2. Giao diện gửi request `GET /api/v1/categories` và `GET /api/v1/products`.
3. Hệ thống trả về danh sách danh mục (Cà phê, Trà) và danh sách 10 sản phẩm (6 món cà phê, 4 món trà theo seed
4. Khách hàng nhấp chọn tab phân loại (ví dụ ”Cà phê”) hoặc nhập từ khóa tìm kiếm (ví dụ ”Latte”).



Hình 5.6: Sơ đồ Hoạt động - Phân hệ Danh mục Sản phẩm



Hình 5.7: Sơ đồ Tuần tự - Phân hệ Danh mục Sản phẩm

Bảng 5.5: Danh mục API Endpoints - Phân hệ Danh mục Sản phẩm

Phương thức	Endpoint	Tham số Query	Phản hồi chính
GET	/api/v1/categories	Không có	200 OK: Danh sách danh mục sắp xếp theo displayOrder tăng dần.
GET	/api/v1/products	categoryId, search, availableOnly	200 OK: Danh sách sản phẩm kèm thông tin giá cơ bản, hình ảnh và trạng thái khả dụng.

- Dữ liệu seed chuẩn gồm 2 danh mục chính (”Cà phê” - 6 món, ”Trà” - 4 món), tổng cộng 10 sản phẩm mẫu được nạp vào bảng categories và products.

5.2.6 Kịch bản Kiểm thử (Test Cases)

Bảng 5.6: Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Danh mục

Mã TC	Loại	Tiền điều kiện	Endpoint / Màn hình	Kết quả mong đợi	Trạng
TC-CAT-API-01	Hợp lệ	DB có 2 danh mục	GET /categories	Trả về đủ 2 danh mục ”Cà phê”, ”Trà”	Chưa c
TC-CAT-API-02	Hợp lệ	DB có 10 sản phẩm	GET /products	Trả về danh sách 10 món kèm basePrice	Chưa c
TC-CAT-UI-01	Giao diện	Trang /products	Thực đơn chính	Hiển thị tab ”Tất cả”, ”Cà phê”, ”Trà”	Chưa c
TC-CAT-UI-07	Giao diện	Chọn tab ”Cà phê”	Lọc danh mục	Hiển thị đúng 6 món cà phê từ seed data	Chưa c

5.2.7 Giao diện Người dùng

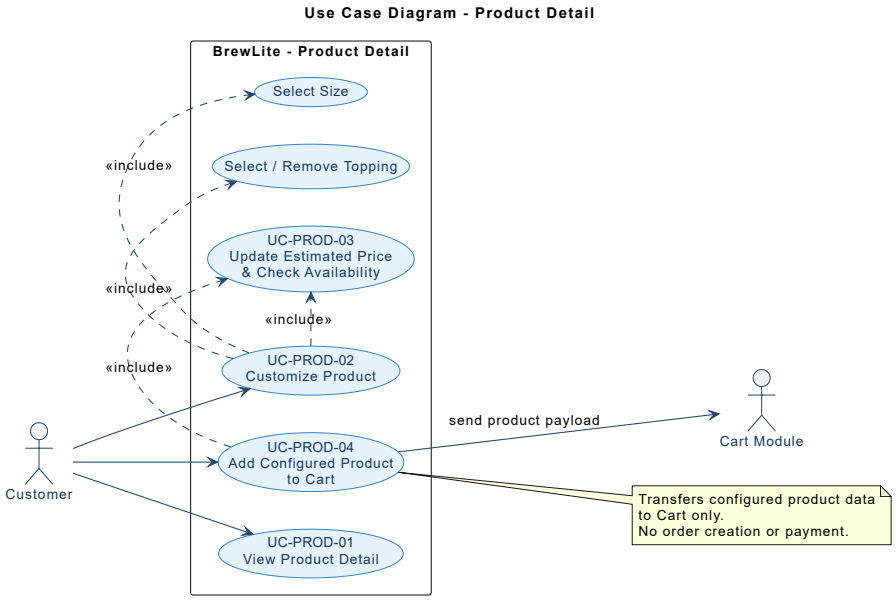
[TODO: Chèn ảnh chụp màn hình Giao diện Thực đơn sản phẩm (/products)]

Hình 5.8: Giao diện Thực đơn sản phẩm BrewLite

5.3 Phân hệ Chi tiết Sản phẩm (Product Detail)

5.3.1 Đặc tả Use Case

Phân hệ Chi tiết Sản phẩm cung cấp thông tin toàn diện về đồ uống, bao gồm mô tả, thành phần dinh dưỡng/nguyên liệu và cho phép khách hàng cá nhân hóa món đồ uống theo sở thích cá nhân. Sơ đồ Use Case được minh họa tại Hình 5.9.



Hình 5.9: Sơ đồ Use Case - Phân hệ Chi tiết Sản phẩm

5.3.2 Sơ đồ Hoạt động (Activity Diagram)

Quy trình người dùng tương tác, chọn lựa tùy biến và kiểm tra trạng thái tồn kho được biểu diễn tại Hình 5.10.

5.3.3 Sơ đồ Tuần tự (Sequence Diagram)

Sơ đồ Tuần tự thể hiện các tương tác truy vấn dữ liệu chi tiết sản phẩm và danh mục topping tại Hình 5.11.

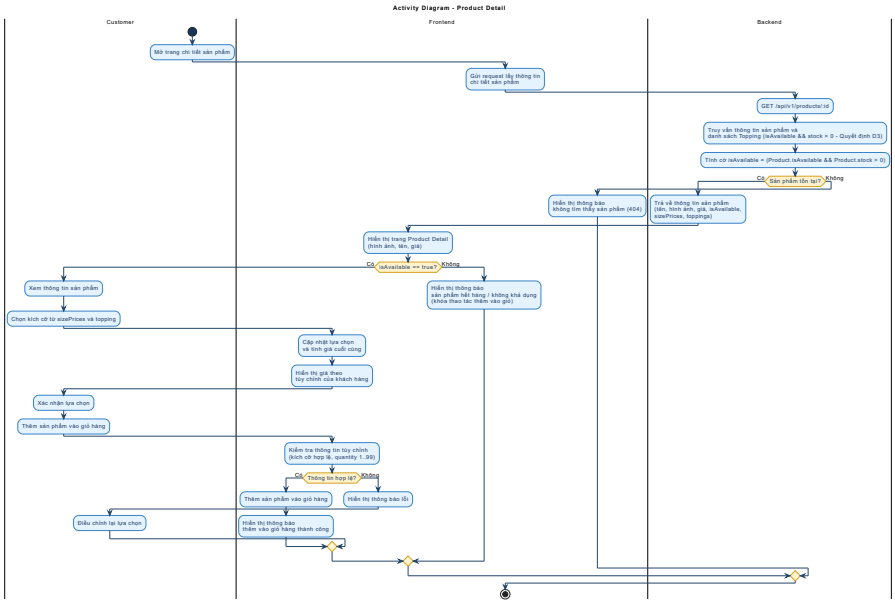
5.3.4 Hợp đồng API (API Contract)

5.3.5 Ánh xạ Cơ sở dữ liệu (Database Mapping)

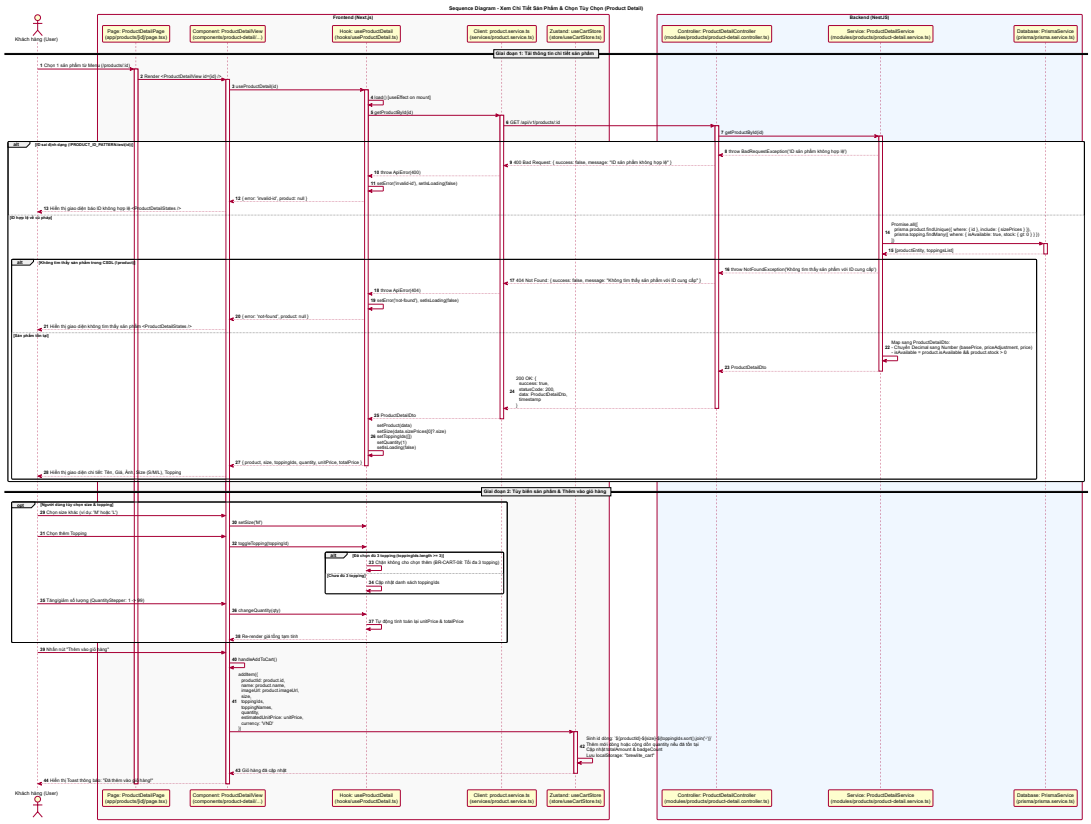
- Truy vấn chi tiết sản phẩm: `prisma.product.findUnique({ where: { id }, include: { sizePrices: true, category: true } })`.
- Truy vấn danh sách topping: `prisma.topping.findMany({ where: { isAvailable: true } })`.

Bảng 5.7: Đặc tả Use Case: Xem chi tiết và Tùy biến món (UC-DET-01)

Mục	
Actor	
Tiền điều kiện	
Hậu điều kiện	



Hình 5.10: Sơ đồ Hoạt động - Phân hệ Chi tiết Sản phẩm



Hình 5.11: Sơ đồ Tuần tự - Phân hệ Chi tiết Sản phẩm

Bảng 5.8: Danh mục API Endpoints - Phân hệ Chi tiết Sản phẩm

Phương thức	Endpoint	Tham số Đường dẫn	Phản hồi chính
`GET`	`/api/v1/products/:id`	`id` (CUID của sản phẩm)	`200 OK`: Chi tiết sản phẩm kèm mảng `sizePrices` và thông tin `category`. Lỗi `404` nếu không tìm thấy.
`GET`	`/api/v1/toppings`	Không có	`200 OK`: Danh sách các loại topping đang hoạt động kèm giá và số lượng tồn.

5.3.6 Kịch bản Kiểm thử (Test Cases)

Bảng 5.9: Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Chi tiết Sản phẩm

Mã TC	Loại	Tiền điều kiện	Endpoint / Màn hình	Kết quả mong đợi
`TC-DET-API-01`	Hợp lệ	ID sản phẩm tồn tại	`GET /products/:id`	HTTP 200, trả về đ sizePrices (S, M, L)
`TC-DET-API-03`	Lỗi	ID không tồn tại	`GET /products/:id`	HTTP 404, messag ”Không tìm thấy sả phẩm”
`TC-DET-UI-01`	Giao diện	Mở trang chi tiết	Màn hình `/products/[id]`	Có nút ”Quay lại thự đơn”, nút ”Thêm vào gi hàng”
`TC-DET-UI-04`	Giao diện	Chọn Size L (+10.000đ)	Tùy biến món	Đơn giá tự động cộn thêm phụ thu tương ứng

5.3.7 Giao diện Người dùng

5.4 Phân hệ Giỏ hàng (Cart)

5.4.1 Đặc tả Use Case

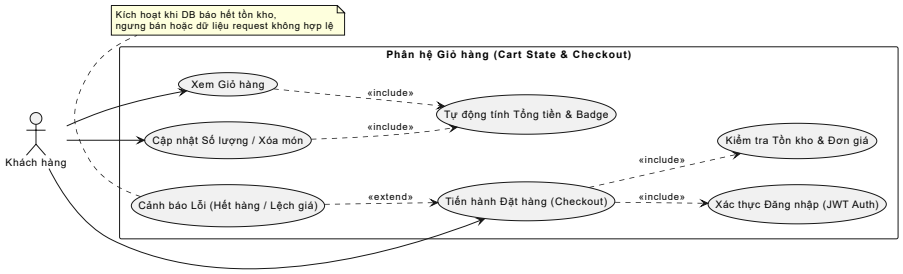
Phân hệ Giỏ hàng quản lý danh sách các món đồ uống mà khách hàng đã lựa chọn kèm theo các tùy biến cụ thể trước khi tiến hành tạo đơn hàng chính thức. Để tối ưu hóa tài nguyên máy chủ và mang lại phản hồi tức thì cho người dùng, phân hệ này được thiết kế theo kiến trúc lưu trữ phía máy khách (*Client-side State Management*). Sơ đồ Use Case được minh họa tại Hình 5.13.

5.4.2 Sơ đồ Hoạt động (Activity Diagram)

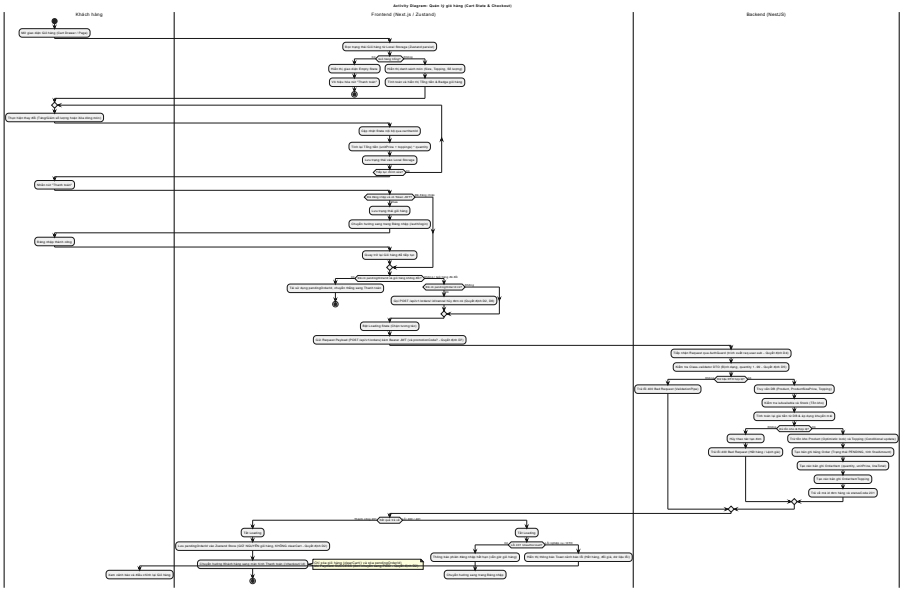
Quy trình quản lý thêm, sửa, xóa và tính toán giá trị giỏ hàng được thể hiện tại Hình 5.14.

[TODO: Chèn ảnh chụp màn hình Giao diện Chi tiết sản phẩm (`/products/[id]`)]

Hình 5.12: Giao diện Tùy biến và Chi tiết sản phẩm



Hình 5.13: Sơ đồ Use Case - Phân hệ Giỏ hàng



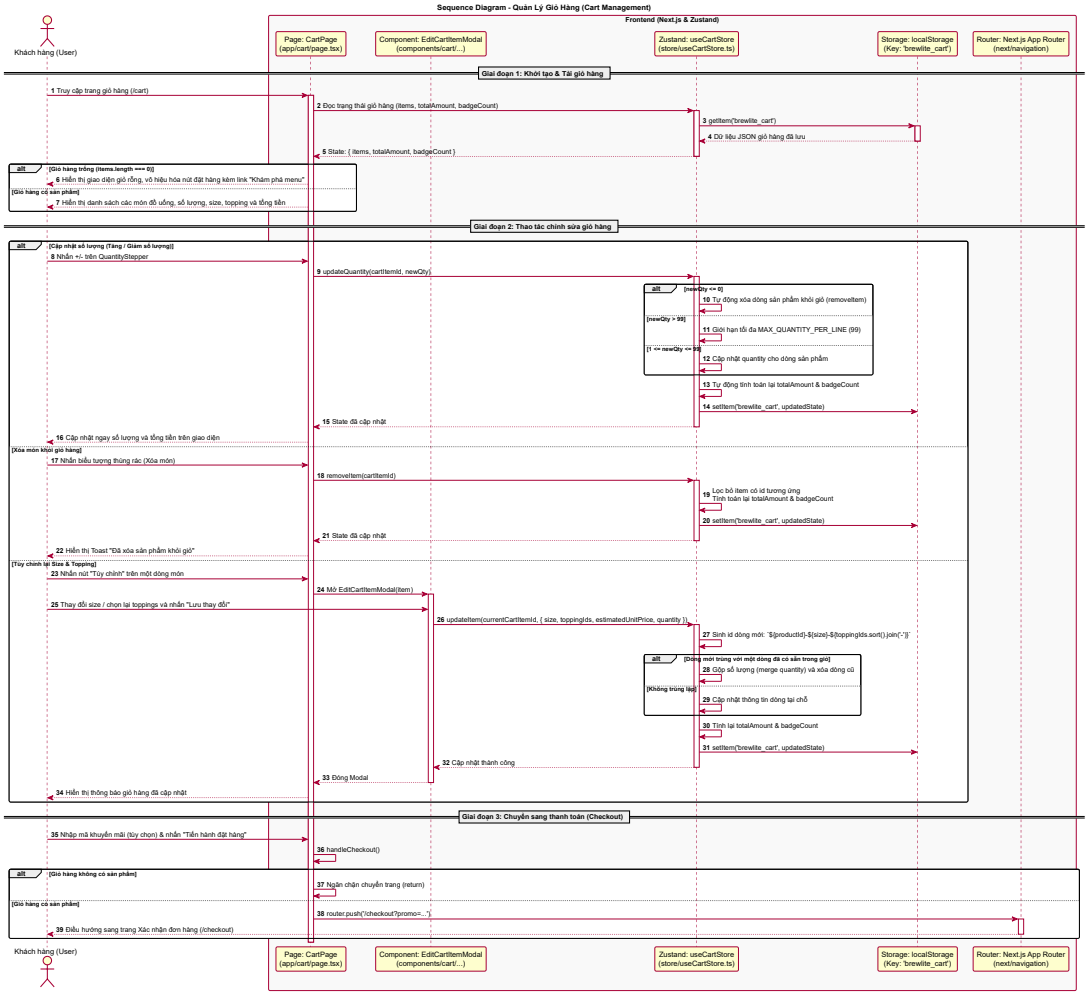
Hình 5.14: Sơ đồ Hoạt động - Phân hệ Giỏ hàng

Bảng 5.10: Đặc tả Use Case: Quản lý Giỏ hàng (UC-CART-01)

Mục	
Actor	
Tiền điều kiện	
Hậu điều kiện	
Luồng chính	

5.4.3 Sơ đồ Tuần tự (Sequence Diagram)

Sơ đồ Tuần tự minh họa luồng xử lý hoàn toàn khép kín ở phía Frontend giữa giao diện, Store quản lý trạng thái và LocalStorage tại Hình 5.15.



Hình 5.15: Sơ đồ Tuần tự - Phân hệ Giỏ hàng (Client-side Architecture)

5.4.4 Hợp đồng API và Ánh xạ Cơ sở dữ liệu

- **Đặc tính kỹ thuật (FE-Only):** Giỏ hàng trong BrewLite hoạt động hoàn toàn ở phía máy khách (*Frontend-only*). Không có API endpoint riêng trên Backend và không tạo bản ghi trong cơ sở dữ liệu cho giỏ hàng tạm thời.
- **Cấu trúc đối tượng lưu trữ (`CartItem`):** Mỗi phần tử giỏ hàng được định danh bởi một khóa duy nhất sinh ra từ tổ hợp thuộc tính (`productId`, `size`, `sweetness`, `ice`, danh sách `toppings`):
  - `id`: Mã duy nhất của món trong giỏ.
  - `productId`: ID sản phẩm tham chiếu.
  - `name`, `image`: Thông tin hiển thị.
  - `size`, `sweetness`, `ice`: Các tùy biến đã chọn.

- `toppings`: Danh sách topping kèm giá đơn vị.
- `quantity`: Số lượng đặt (≥ 1).
- `unitPrice`: Đơn giá đã bao gồm phụ thu kích cỡ.
- `subtotal`: Tổng tiền của dòng món = $(unitPrice + \sum toppingPrice) \times quantity$.

5.4.5 Kịch bản Kiểm thử (Test Cases)

Bảng 5.11: Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Giỏ hàng

Mã TC	Loại	Tiền điều kiện	Endpoint / Màn hình	Kết quả mong đợi	Trạng th
`TC-CART-UI-01`	Giao diện	Giỏ chưa có món	Màn hình `/cart`	Hiển thị thông báo giỏ hàng trống	Chưa cha
`TC-CART-UI-02`	Nghiệp vụ	Đã thêm 1 món	Màn hình `/cart`	Hiển thị đúng tên, size, mức đường, đá, topping	Chưa cha
`TC-CART-UI-03`	Nghiệp vụ	Đang có món	Nút tăng/giảm	Cập nhật số lượng và tự động tính lại tổng tiền	Chưa cha
`TC-CART-UI-04`	Nghiệp vụ	Đang có món	Nút xóa món	Món bị loại bỏ khỏi danh sách, tính lại tiền	Chưa cha

5.4.6 Giao diện Người dùng

[TODO: Chèn ảnh chụp màn hình Giao diện Giỏ hàng (`/cart`)]

Hình 5.16: Giao diện Quản lý Giỏ hàng BrewLite

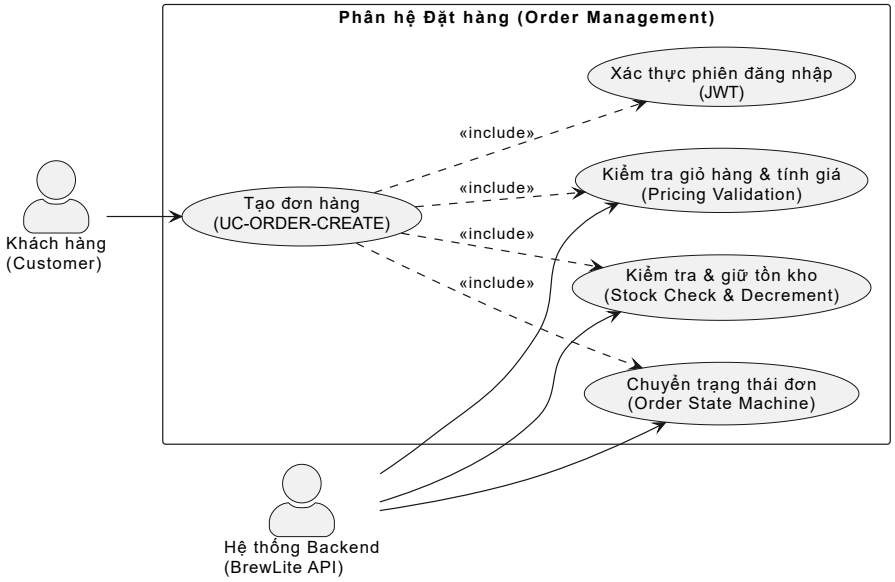
5.5 Phân hệ Đặt hàng (Order)

5.5.1 Đặc tả Use Case

Phân hệ Đặt hàng là hạt nhân nghiệp vụ của hệ thống BrewLite, chịu trách nhiệm chuyển hóa các sản phẩm trong giỏ hàng thành đơn hàng chính thức. Phân hệ tích hợp các dịch vụ tính giá (`PricingService`), kiểm tra và áp dụng khuyến mãi (`PromotionsService`), quy đổi điểm thành viên (`LoyaltyService`) và kiểm soát kho bằng khóa lạc quan (`Optimistic Locking`). Sơ đồ Use Case được minh họa tại Hình 5.17.

Bảng 5.12: Đặc tả Use Case: Tạo đơn hàng trực tuyến (UC-ORD-01)

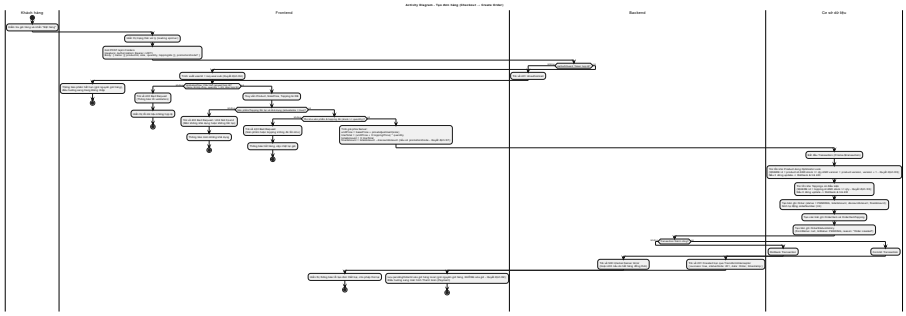
Mục	**Chi tiết Đặc tả**
Actor	Khách hàng đã đăng nhập (Registered Customer)
Tiền điều kiện	Khách hàng đã đăng nhập (mang Bearer JWT Token); giỏ hàng có ít nhất 1 sản phẩm
Hậu điều kiện	Bản ghi đơn hàng mới được tạo ở trạng thái `PENDING`; tồn kho sản phẩm được cập nhật; voucher được áp dụng



Hình 5.17: Sơ đồ Use Case - Phân hệ Đặt hàng

5.5.2 Sơ đồ Hoạt động (Activity Diagram)

Luồng hoạt động kiểm tra điều kiện tồn kho, tính giá và cam kết giao dịch tạo đơn được biểu diễn tại Hình 5.18.



Hình 5.18: Sơ đồ Hoạt động - Phân hệ Đặt hàng

5.5.3 Sơ đồ Tuần tự (Sequence Diagram)

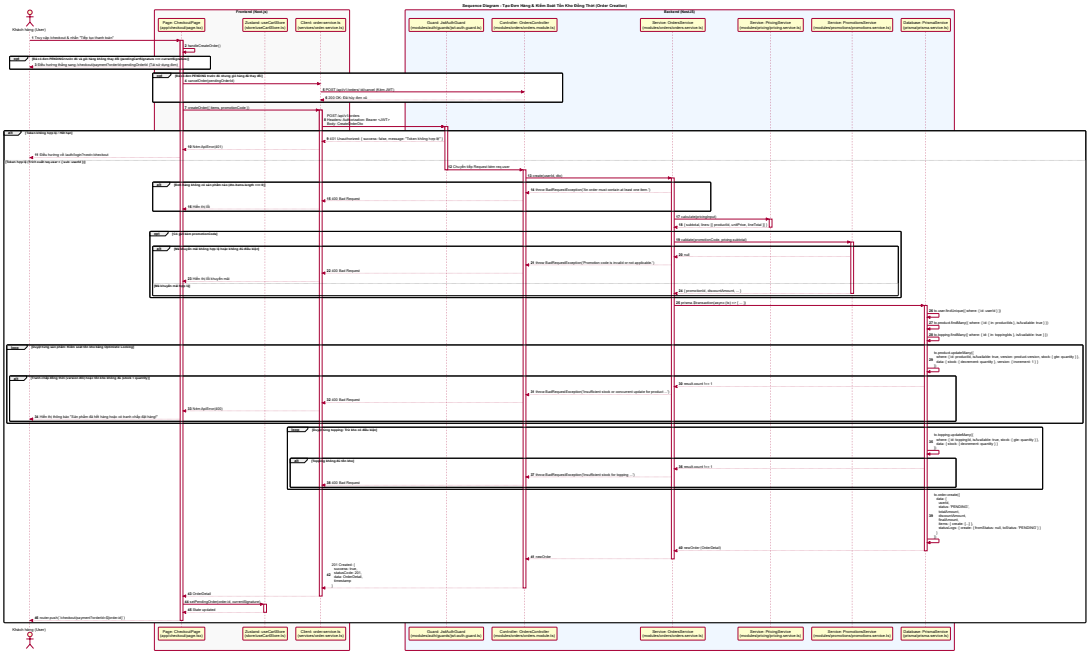
Sơ đồ Tuần tự minh họa chi tiết toàn bộ các lời gọi dịch vụ phối hợp trong một transaction tạo đơn hàng tại Hình 5.19.

5.5.4 Hợp đồng API (API Contract)

5.5.5 Ánh xạ Cơ sở dữ liệu (Database Mapping)

Quá trình tạo đơn hàng thực thi nguyên tử trong một `prisma.$transaction`:

```
-- 1. Kiem tra va cap nhat ton kho voi Optimistic Locking
UPDATE products
SET stock = stock - :quantity, version = version + 1
```



Hình 5.19: Sơ đồ Tuần tự - Phân hệ Đặt hàng (Interactive Transaction & Pricing Engine)

Bảng 5.13: Danh mục API Endpoints - Phân hệ Đặt hàng

Phương thức	**Endpoint**	**Mô tả Request Payload**	**Phản hồi chính**
`POST`	`/api/v1/orders`	`{items: [{productId, size, quantity, toppings}], voucherCode?, pointsToRedeem?}` (Header: `Authorization: Bearer <JWT>`)	`201 Created`: Trả về đơn hàng mới tạo với cấu trúc bọc chuẩn `TransformInterceptor`. Lỗi: 400, 401, 409.
`GET`	`/api/v1/orders/:id`	Tham số `id` của đơn hàng (Header: Bearer Token)	`200 OK`: Chi tiết đơn hàng, danh sách món, trạng thái xử lý và các lần thử thanh toán.

```
WHERE id = :productId AND version = :currentVersion AND stock >= :quantity;

-- 2. Tao ban ghi don hang chinh thuc
INSERT INTO orders (id, order_number, user_id, status, total_amount,
    discount_amount, final_amount, points_earned, version, created_at,
    updated_at)
VALUES (cuid(), nextval('orders_order_number_seq'), :userId, 'PENDING', :
    totalAmount, :discountAmount, :finalAmount, :pointsEarned, 0, NOW(), NOW
    ());

-- 3. Tao cac chi tiet don hang va luu snapshot gia
INSERT INTO order_items (id, order_id, product_id, size, quantity,
    unit_price, line_total)
VALUES (cuid(), :orderId, :productId, :size, :quantity, :unitPrice, :
    lineTotal);

-- 4. Ghi nhat ky trang thai ban dau
INSERT INTO order_status_history (id, order_id, from_status, to_status,
    changed_by, created_at)
VALUES (cuid(), :orderId, NULL, 'PENDING', 'system', NOW());
```

5.5.6 Kịch bản Kiểm thử (Test Cases)

Bảng 5.14: Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Đặt hàng

Mã TC	Loại	Tiền điều kiện	Endpoint / Màn hình	Kết quả mong đợi
TC-ORD-API-01	Hợp lệ	Đủ tồn kho, có token	POST /orders	HTTP 201, tạo đơn trạng thái PENDING, trừ kho
TC-ORD-API-03	Lỗi	Tồn kho không đủ	POST /orders	HTTP 400, rollback transaction, không trừ kho
TC-ORD-API-04	Lỗi	Sai version (Optimistic)	POST /orders	HTTP 409 Conflict, ném lỗi xung đột
TC-ORD-API-06	Hợp lệ	Áp voucher WELCOME10	POST /orders	Giảm 10% (tối đa 50k), finalAmount chính xác
TC-ORD-UI-01	Giao diện	Đã đăng nhập	Trang /checkout	Hiển thị tóm tắt đơn, ô nhập voucher và nút đặt

5.5.7 Giao diện Người dùng

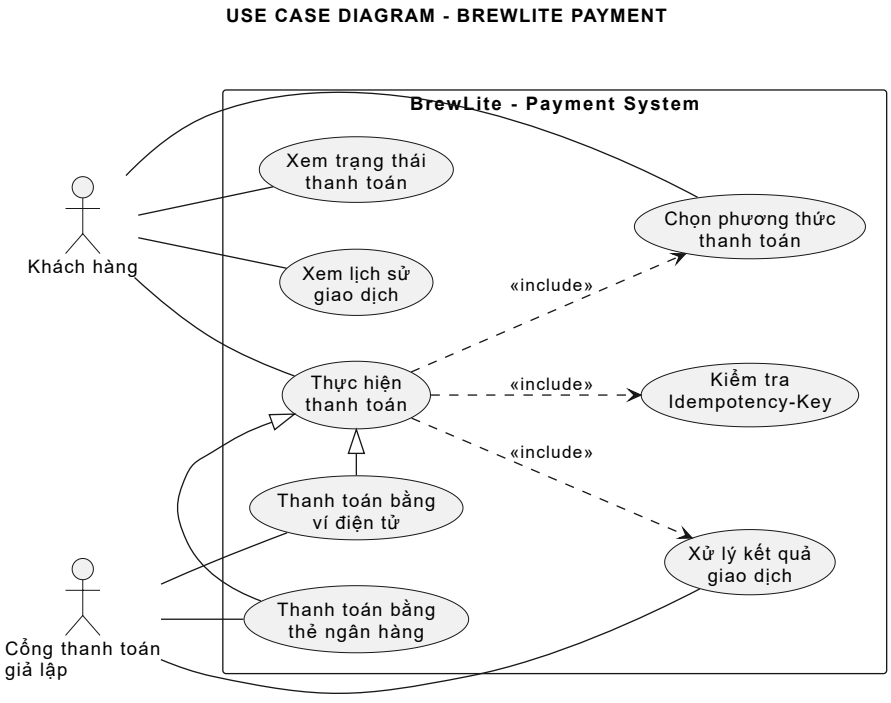
5.6 Phân hệ Thanh toán (Payment)

5.6.1 Đặc tả Use Case

Phân hệ Thanh toán chịu trách nhiệm tiếp nhận và xử lý giao dịch tài chính cho các đơn hàng đang ở trạng thái chờ. Phân hệ được thiết kế với cơ chế an toàn cấp cao: bảo vệ giao dịch bất biến (*Idempotency*) để chống trùng lặp tiền khi người dùng nhấp đúp hoặc xảy ra lỗi chập chờn mạng, đồng thời hỗ trợ nhiều lượt thử lại (*Retry*) cho cùng một đơn hàng thông qua quan hệ 1-N giữa `Order` và `Payment`. Sơ đồ Use Case được minh họa tại Hình 5.21.

[TODO: Chèn ảnh chụp màn hình Giao diện Đặt hàng (`/checkout`)]

Hình 5.20: Giao diện Xác nhận Đặt hàng và Áp dụng Khuyến mãi



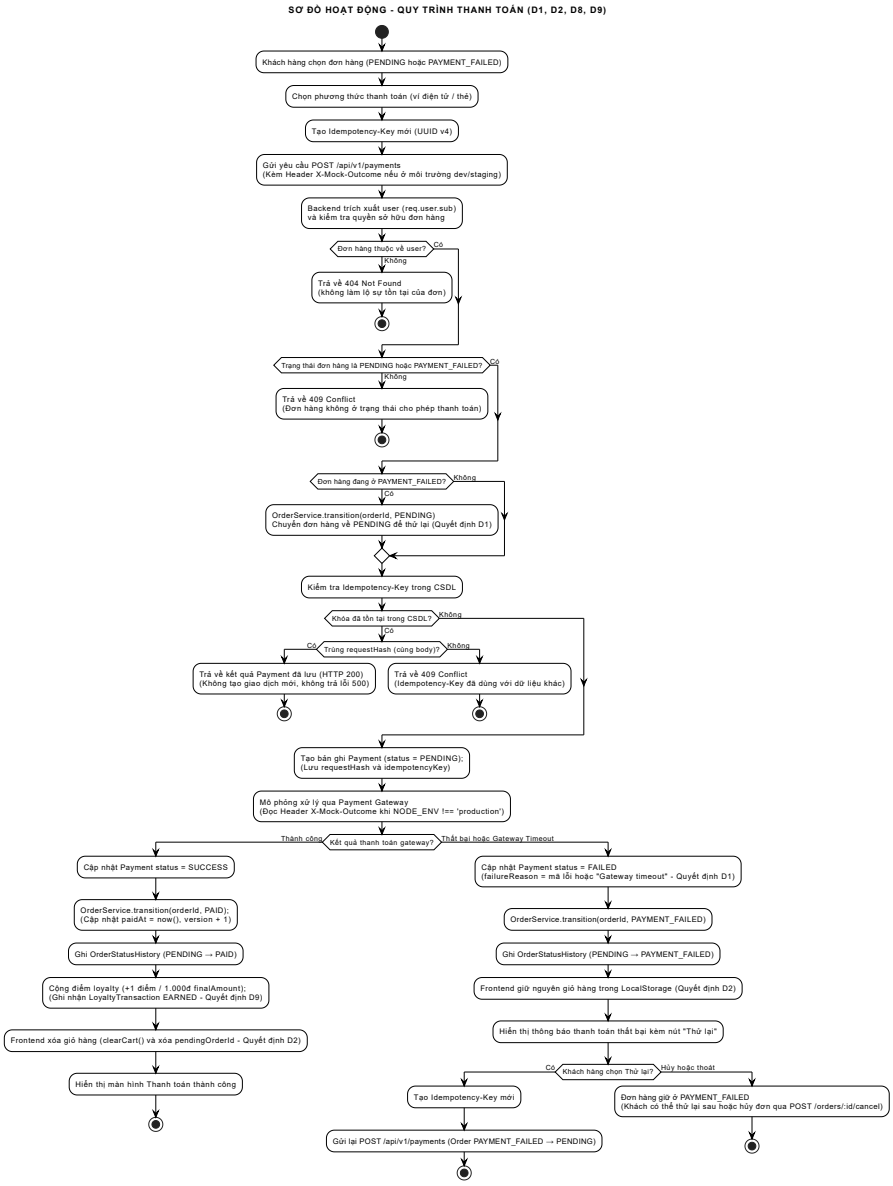
Hình 5.21: Sơ đồ Use Case - Phân hệ Thanh toán

Bảng 5.15: Đặc tả Use Case: Thực hiện Thanh toán Trực tuyến (UC-PAY-01)

Mục	
Actor	
Tiền điều kiện	
Hậu điều kiện	

5.6.2 Sơ đồ Hoạt động (Activity Diagram)

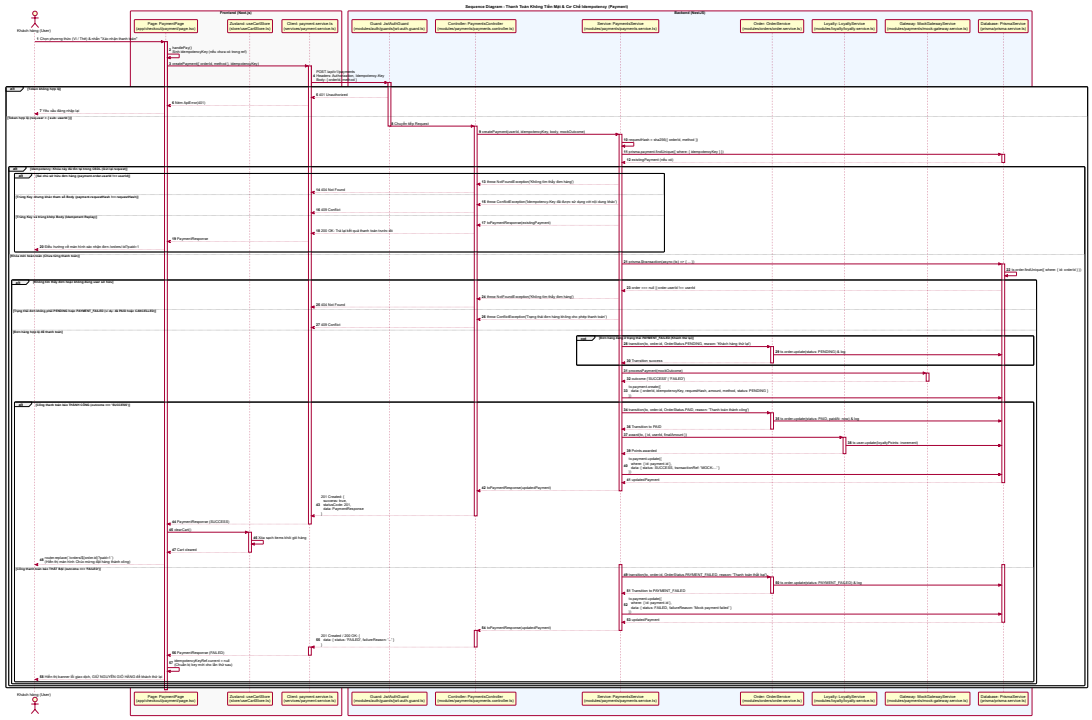
Quy trình thẩm định Idempotency, gọi cổng thanh toán và xử lý kết quả giao dịch được mô tả tại Hình 5.22.



Hình 5.22: Sơ đồ Hoạt động - Phân hệ Thanh toán

5.6.3 Sơ đồ Tuần tự (Sequence Diagram)

Sơ đồ Tuần tự phản ánh toàn diện các luồng tương tác giữa Frontend, PaymentsController, PaymentsService, Prisma DB và Mock Payment Gateway tại Hình 5.23.



Hình 5.23: Sơ đồ Tuần tự - Phân hệ Thanh toán (Idempotent Flow & Gateway Integration)

Bảng 5.16: Danh mục API Endpoints - Phân hệ Thanh toán

Phương thức	**Endpoint**	**Headers & Body**	**Phản hồi chính**
`POST`	`/api/v1/payments`	Header: `Idempotency-Key: <UUID>`, Bearer Token. Body: `{orderId, paymentMethod}`	`200 OK` / `201 Created`: Trả về chi tiết thanh toán thành công. Lỗi: `400`, `401`, `404`, `409 Conflict`.
`GET`	`/api/v1/payments/:id`	Tham số `id` bản ghi thanh toán	`200 OK`: Tra cứu trạng thái giao dịch thanh toán hiện tại.

5.6.4 Hợp đồng API (API Contract)

5.6.5 Ánh xạ Cơ sở dữ liệu (Database Mapping)

- **Cơ chế Idempotency:** Bảng `payments` sử dụng ràng buộc `@unique` trên trường `idempotency_key` kết hợp cùng `request_hash` sinh từ thuật toán SHA-256 trên nội dung JSON của body request:

```
SELECT id, status, request_hash, transaction_ref
FROM payments
WHERE idempotency_key = :idempotencyKey;
```

- **Cập nhật sau thanh toán thành công:** Thực thi trong transaction cập nhật đồng thời 4 bảng:

```
UPDATE payments SET status = 'SUCCESS', transaction_ref = :ref WHERE
    id = :paymentId;
UPDATE orders SET status = 'PAID', paid_at = NOW() WHERE id = :orderId
    ;
UPDATE users SET loyalty_points = loyalty_points + :pointsEarned WHERE
     id = :userId;
INSERT INTO loyalty_transactions (id, user_id, order_id, points, type,
     reason, created_at)
VALUES (cuid(), :userId, :orderId, :pointsEarned, 'EARNED', 'Tich diem
     tu don hang', NOW());
INSERT INTO order_status_history (id, order_id, from_status, to_status
    , changed_by, created_at)
VALUES (cuid(), :orderId, 'PENDING', 'PAID', 'payment_gateway', NOW())
    ;
```

5.6.6 Kịch bản Kiểm thử (Test Cases)

Bảng 5.17: Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Thanh toán

Mã TC	Loại	Tiền điều kiện	Endpoint / Màn hình	Kết quả mong đợi
TC-PAY-API-01	Hợp lệ	Đơn PENDING, key mới	POST /payments	HTTP 200, status SUCCESS, đơn chuyển sang PAID
TC-PAY-API-03	Idempotent	Gửi lại cùng key	POST /payments	HTTP 200, trả về kết quả cũ, không gọi lại gateway
TC-PAY-API-05	Lỗi	Đơn đã thanh toán	POST /payments	HTTP 400, báo lỗi đơn hàng đã thanh toán
TC-PAY-UI-01	Giao diện	Đơn hàng chờ	Trang /payment/[id]	Hiển thị tổng tiền và 2 phương thức thẻ/ví

[TODO: Chèn ảnh chụp màn hình Giao diện Thanh toán trực tuyến (`/payment/[orderId]`)]

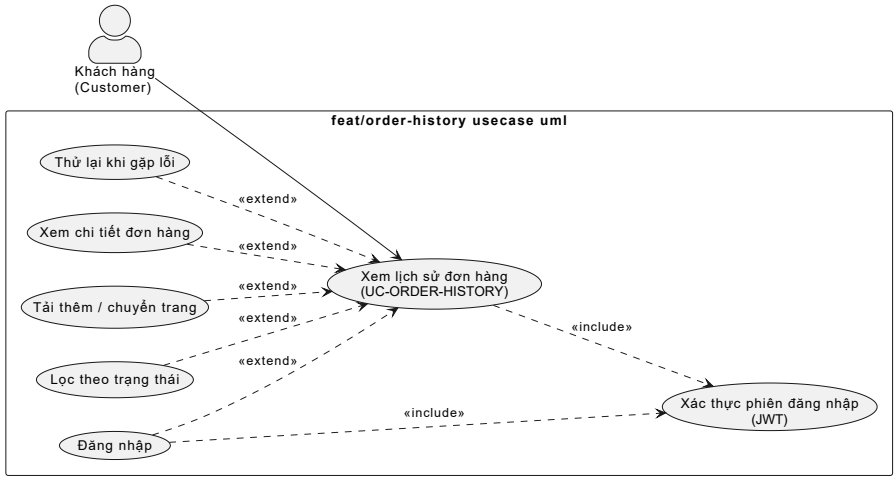
Hình 5.24: Giao diện Chọn Phương thức Thanh toán

5.6.7 Giao diện Người dùng

5.7 Phân hệ Lịch sử Đơn hàng (Order History)

5.7.1 Đặc tả Use Case

Phân hệ Lịch sử Đơn hàng cho phép khách hàng tra cứu toàn bộ các đơn hàng đã đặt trong quá khứ, theo dõi trạng thái pha chế và kiểm tra hóa đơn chi tiết. Sơ đồ Use Case được minh họa tại Hình 5.25.



Hình 5.25: Sơ đồ Use Case - Phân hệ Lịch sử Đơn hàng

5.7.2 Sơ đồ Hoạt động (Activity Diagram)

Quy trình truy vấn danh sách và xem chi tiết trạng thái đơn hàng được thể hiện tại Hình 5.26.

5.7.3 Sơ đồ Tuần tự (Sequence Diagram)

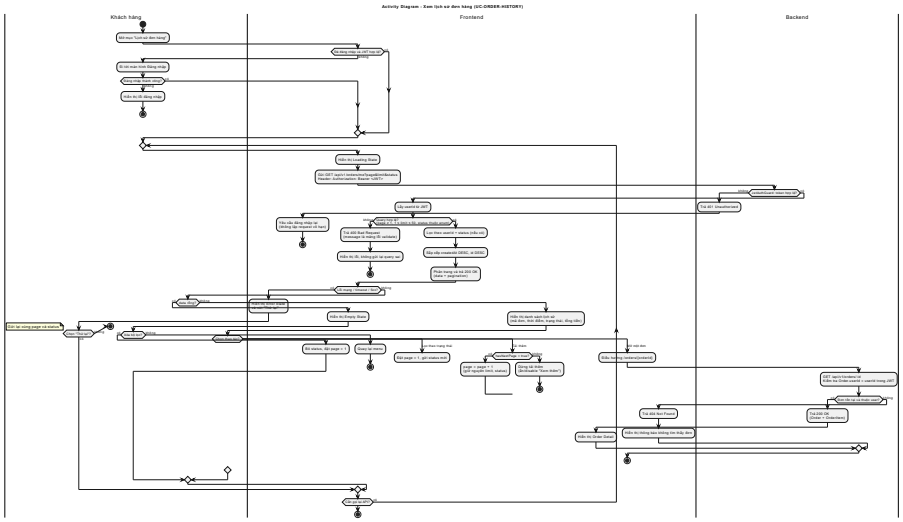
Sơ đồ Tuần tự minh họa tương tác nạp dữ liệu lịch sử đơn hàng từ Frontend đến Controller, Service và Prisma tại Hình 5.27.

Bảng 5.18: Đặc tả Use Case: Tra cứu Lịch sử và Chi tiết Đơn hàng (UC-HIS-01)

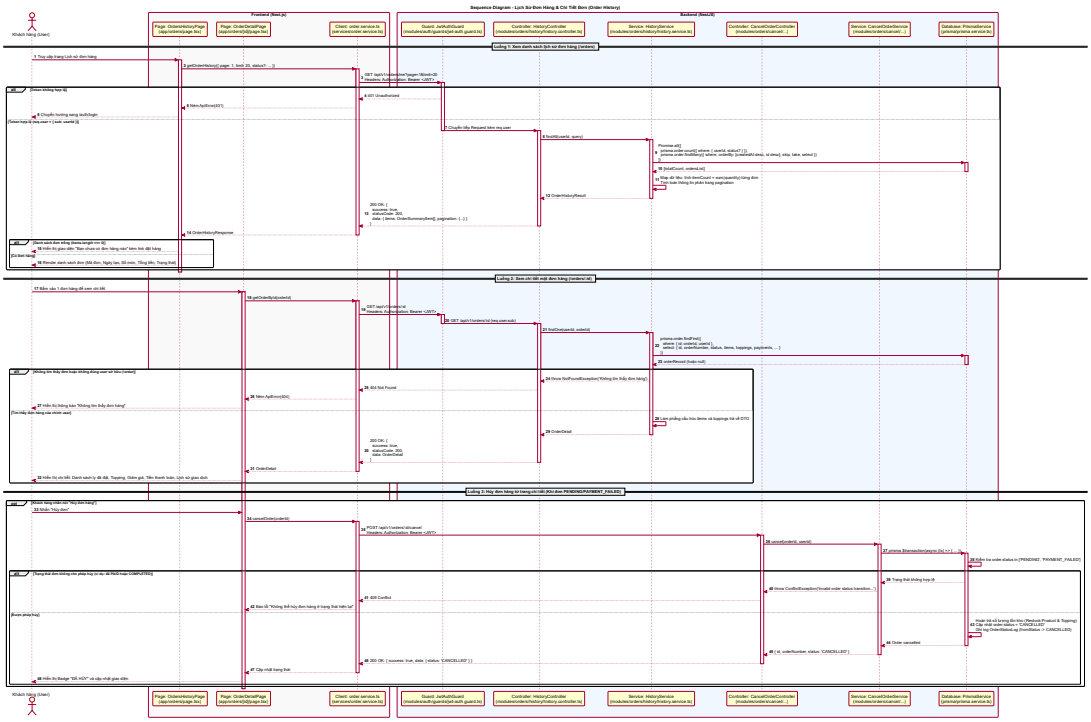
Mục	
Actor	
Tiền điều kiện	
Hậu điều kiện	
Luồng chính	

2. Giao diện gửi request `GET /api/v1/orders` kèm Bearer Token.
3. Backend truy vấn các đơn hàng thuộc về `userId` hiện tại, sắp xếp giảm dần theo thời gian tạo (`createdAt des`
4. Màn hình hiển thị danh sách các thẻ đơn hàng: mã hiển thị `#orderNumber`, ngày giờ đặt, tổng tiền, huy hiệu trạ
5. Khách hàng nhấp vào một đơn hàng để xem chi tiết hóa đơn (`/orders/[id]`): danh sách từng món, size, toppin

Luồng ngoại lệ



Hình 5.26: Sơ đồ Hoạt động - Phân hệ Lịch sử Đơn hàng



Hình 5.27: Sơ đồ Tuần tự - Phân hệ Lịch sử Đơn hàng

5.7.4 Hợp đồng API (API Contract)

Bảng 5.19: Danh mục API Endpoints - Phân hệ Lịch sử Đơn hàng

Phương thức	Endpoint	Header	Phản hồi chính
`GET`	`/api/v1/orders`	`Authorization: Bearer <JWT>`	`200 OK`: Danh sách các đơn hàng của user đang đăng nhập, kèm tổng tiền và trạng thái.
`GET`	`/api/v1/orders/:id`	`Authorization: Bearer <JWT>`	`200 OK`: Chi tiết đơn hàng gồm các món (`items`), topping và lịch sử trạng thái (`statusHistory`). Lỗi `404` nếu không tìm thấy hoặc không thuộc sở hữu.

5.7.5 Ánh xạ Cơ sở dữ liệu (Database Mapping)

- Truy vấn danh sách đơn hàng cá nhân:

```
SELECT id, order_number, status, total_amount, discount_amount,
    final_amount, created_at
FROM orders
WHERE user_id = :userId
ORDER BY created_at DESC;
```

- Tối ưu hóa: Chỉ mục `@@index([userId])` và `@@index([status])` trên bảng `orders` đảm bảo câu truy vấn thực thi với độ phức tạp $\mathcal{O}(\log N)$, đáp ứng yêu cầu phản hồi nhanh của hệ thống.

5.7.6 Kịch bản Kiểm thử (Test Cases)

Bảng 5.20: Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Lịch sử

Mã TC	Loại	Tiền điều kiện	Endpoint / Màn hình	Kết quả mong đợi	Tr
`TC-HIS-API-01`	Hợp lệ	User có 3 đơn hàng	`GET /orders`	HTTP 200, trả về đủ 3 đơn sắp xếp mới nhất	Ch
`TC-HIS-API-02`	Bảo mật	Xem đơn của user khác	`GET /orders/:id`	HTTP 404 hoặc 403, không lộ đơn của người khác	Ch
`TC-HIS-UI-01`	Giao diện	User chưa có đơn nào	Màn hình `/orders`	Hiển thị chuỗi ”Bạn chưa có đơn hàng nào”	Ch
`TC-HIS-UI-03`	Giao diện	Đơn đã bị hủy	Danh sách đơn	Hiển thị đúng badge trạng thái ”Đã hủy”	Ch

5.7.7 Giao diện Người dùng

[TODO: Chèn ảnh chụp màn hình Giao diện Lịch sử đơn hàng (`/orders`)]

Hình 5.28: Giao diện Lịch sử Đơn hàng của Khách hàng

Chương 6

HIỆN THỰC CÁC NGHIỆP VỤ NÂNG CAO

Chương này đi sâu vào các giải pháp kỹ thuật phức tạp đã được thiết kế và hiện thực hóa trong hệ thống BrewLite nhằm giải quyết các bài toán hóc búa về tính toàn vẹn dữ liệu, kiểm soát đồng thời và độ tin cậy giao dịch trong môi trường đa người dùng.

6.1 Máy trạng thái Đơn hàng (Order State Machine)

Quy trình luân chuyển trạng thái của một đơn hàng trong BrewLite được quản lý chặt chẽ thông qua mô hình Máy trạng thái hữu hạn (*Finite State Machine*), định nghĩa nguyên văn trong tập tin mã nguồn `backend/src/modules/orders/order-state.ts`.

Sơ đồ máy trạng thái đơn hàng được trình bày chi tiết tại Hình 6.1.

Hệ thống quy định chính xác **9 chuyển tiếp trạng thái hợp lệ** giữa 7 trạng thái nghiệp vụ:

1. `PENDING` → `PAID`: Chuyển đổi khi khách hàng thực hiện thanh toán trực tuyến thành công qua cổng thanh toán.
2. `PENDING` → `PAYMENT_FAILED`: Chuyển đổi khi giao dịch thanh toán bị từ chối, thẻ hết hạn hoặc phát sinh lỗi mạng.
3. `PENDING` → `CANCELLED`: Khách hàng chủ động hủy đơn khi chưa thanh toán hoặc đơn hàng hết thời gian chờ (timeout).
4. `PAID` → `PREPARING`: Quầy pha chế (Barista) tiếp nhận đơn hàng đã thanh toán và bắt đầu quy trình làm đồ uống.
5. `PAID` → `CANCELLED`: Đơn hàng đã thanh toán bị hủy do sự cố bất khả kháng (ví dụ hết nguyên liệu đột xuất), kích hoạt luồng hoàn tiền.
6. `PREPARING` → `READY`: Nhân viên pha chế hoàn tất đồ uống, đơn hàng sẵn sàng chờ khách hàng đến nhận tại quầy.
7. `READY` → `COMPLETED`: Khách hàng nhận đồ uống thành công. Đây là một trong hai trạng thái kết thúc (*Terminal State*).
8. `PAYMENT_FAILED` → `PENDING`: Khách hàng nhấn thử lại (*Retry Payment*), chọn phương thức thanh toán mới để tiến hành trả tiền lại cho đơn hàng.
9. `PAYMENT_FAILED` → `CANCELLED`: Đơn hàng bị hủy vĩnh viễn sau khi thanh toán thất bại và khách hàng không có nhu cầu thử lại.

Hai trạng thái `COMPLETED` và `CANCELLED` là trạng thái kết thúc vĩnh viễn (*Terminal States*), danh sách chuyển tiếp tiếp theo là mảng rỗng (`[]`). Bất kỳ nỗ lực chuyển đổi sai quy tắc nào đều bị chặn ngay lập tức bởi hàm kiểm tra `assertTransition()` và ném ngoại lệ `409 ConflictException`:

An error has occured : net.sourceforge.plantuml.vizjs.GraphvizJsRuntimeException: java.util.concurrent.ExecutionException: java.lang.NullPointerException: Cannot invoke "net.sourceforge.plantuml.vizjs.VizJsEngine.execute(String)" because "engine" is null
My reality is just different than yours

Diagram size: 52 lines / 2166 characters.

PlantUML (1.2019.06) cannot parse result from dot/GraphViz.
This version of PlantUML is 2710 days old, so you should
consider upgrading from http://plantuml.com/download

Please go to http://plantuml.com/graphviz-dot to check your GraphViz version.

Java Runtime: Java(TM) SE Runtime Environment
JVM: Java HotSpot(TM) 64-Bit Server VM
Java Version: 21.0.12.1+1-LTS-4
Operating System: Windows 11
OS Version: 10.0
Default Encoding: UTF-8
Language: en
Country: US
Machine: LeoPham
PLANTUML_LIMIT_SIZE: 4096
Processors: 12
Max Memory: 4,255,121,408
Total Memory: 266,338,304
Free Memory: 193,664,856
Used Memory: 72,673,448
Thread Active Count: 2

This may be caused by :
- a bug in PlantUML
- a problem in GraphViz

You should send this diagram and this image to **plantuml@gmail.com** or
post to **http://plantuml.com/qa** to solve this issue.
You can try to turn arround this issue by simplifing your diagram.

net.sourceforge.plantuml.vizjs.GraphvizJsRuntimeException: java.util.concurrent.ExecutionException: java.lang.NullPointerException: Cannot invoke "net.sourceforge.plantuml.vizjs.VizJsEngine.execute(String)" because "engine" is null
net.sourceforge.plantuml.vizjs.GraphvizJs.createFile3(GraphvizJs.java:96)
net.sourceforge.plantuml.svek.DotStringFactory.getSvg(DotStringFactory.java:315)
net.sourceforge.plantuml.svek.GeneralImageBuilder.buildImage(GeneralImageBuilder.java:199)
net.sourceforge.plantuml.svek.CucaDiagramFileMakerSvek.createFileInternal(CucaDiagramFileMakerSvek.java:105)
net.sourceforge.plantuml.svek.CucaDiagramFileMakerSvek.createFile(CucaDiagramFileMakerSvek.java:71)
net.sourceforge.plantuml.cucadiagram.CucaDiagram.exportDiagramInternal(CucaDiagram.java:379)
net.sourceforge.plantuml.UmlDiagram.exportDiagramNow(UmlDiagram.java:202)
net.sourceforge.plantuml.AbstractPSystem.exportDiagram(AbstractPSystem.java:135)
net.sourceforge.plantuml.SourceStringReader.outputImage(SourceStringReader.java:154)
net.sourceforge.plantuml.Pipe.managePipe(Pipe.java:110)
net.sourceforge.plantuml.Run.managePipe(Run.java:359)
net.sourceforge.plantuml.Run.main(Run.java:166)

Caused by java.util.concurrent.ExecutionException: java.lang.NullPointerException: Cannot invoke "net.sourceforge.plantuml.vizjs.VizJsEngine.execute(String)" because "engine" is null
java.base/java.util.concurrent.FutureTask.report(FutureTask.java:122)
java.base/java.util.concurrent.FutureTask.get(FutureTask.java:191)
net.sourceforge.plantuml.vizjs.GraphvizJs.createFile3(GraphvizJs.java:91)
net.sourceforge.plantuml.svek.DotStringFactory.getSvg(DotStringFactory.java:315)
net.sourceforge.plantuml.svek.GeneralImageBuilder.buildImage(GeneralImageBuilder.java:199)
net.sourceforge.plantuml.svek.CucaDiagramFileMakerSvek.createFileInternal(CucaDiagramFileMakerSvek.java:105)
net.sourceforge.plantuml.svek.CucaDiagramFileMakerSvek.createFile(CucaDiagramFileMakerSvek.java:71)
net.sourceforge.plantuml.cucadiagram.CucaDiagram.exportDiagramInternal(CucaDiagram.java:379)
net.sourceforge.plantuml.UmlDiagram.exportDiagramNow(UmlDiagram.java:202)
net.sourceforge.plantuml.AbstractPSystem.exportDiagram(AbstractPSystem.java:135)
net.sourceforge.plantuml.SourceStringReader.outputImage(SourceStringReader.java:154)
net.sourceforge.plantuml.Pipe.managePipe(Pipe.java:110)
net.sourceforge.plantuml.Run.managePipe(Run.java:359)
net.sourceforge.plantuml.Run.main(Run.java:166)

Diagram source: (Use http://zxing.org/w/decode.jspx to decode the qrcode)

Hình 6.1: Sơ đồ Máy trạng thái Đơn hàng BrewLite (9 chuyển tiếp hợp lệ)

```
export function assertTransition(fromStatus: OrderStatus, toStatus:
    OrderStatus): void {
  const allowedStatuses = ALLOWED_TRANSITIONS[fromStatus];
  if (!allowedStatuses?.includes(toStatus)) {
    throw new ConflictException(
      `Invalid order status transition: ${fromStatus} -> ${toStatus}`
    );
  }
}
```

6.2 Xử lý Thanh toán Bất biến (Idempotent Payment)

Trong môi trường mạng không ổn định, khách hàng thường có xu hướng nhấp đúp vào nút "Thanh toán" hoặc trình duyệt tự động gửi lại request khi mạng chập chờn (*Network Timeout*). Nếu không có cơ chế bảo vệ, người dùng sẽ bị trừ tiền nhiều lần cho cùng một đơn hàng (*Double Charge*).

BrewLite giải quyết triệt để vấn đề này bằng cơ chế **Idempotency** cấp độ doanh nghiệp:

1. **Sinh khóa định danh duy nhất (`Idempotency-Key`):** Mỗi khi người dùng bấm xác nhận thanh toán, client tự động sinh một mã UUIDv4 duy nhất và đính kèm vào HTTP Header: `Idempotency-Key: <UUID>`.

2. **Mã hóa toàn vẹn dữ liệu (`request_hash`):** Backend nhận request và tính toán mã băm SHA-256 trên nội dung JSON của request body. Mã băm này đại diện cho tính nguyên vẹn của giao dịch.

3. **Kiểm tra và Điều phối giao dịch:**

   - **Trường hợp 1 (Trùng lặp thành công - Replay):** Nếu `idempotency_key` đã tồn tại trong bảng `payments` và có trạng thái `SUCCESS`, hệ thống lập tức trả về kết quả đã lưu trước đó mà không thực hiện gọi sang Mock Gateway.
   - **Trường hợp 2 (Xung đột đồng thời - Concurrent Request):** Nếu `idempotency_key` đang ở trạng thái `PENDING` (giao dịch trước đó đang xử lý dở dang), hệ thống lập tức từ chối và ném lỗi `409 Conflict`.
   - **Trường hợp 3 (Sai lệch tải trọng - Payload Tampering):** Nếu `idempotency_key` trùng nhưng `request_hash` khác nhau (ví dụ cố tình gửi cùng key nhưng đổi số tiền hoặc ID đơn hàng), hệ thống ném lỗi `422 Unprocessable Entity`.
   - **Trường hợp 4 (Giao dịch mới):** Bản ghi thanh toán được tạo với trạng thái `PENDING`, gọi cổng thanh toán, sau đó cập nhật kết quả cuối cùng.

6.3 Kiểm soát Đồng thời Tồn kho (Concurrency Control)

Một trong những thách thức lớn nhất của các ứng dụng thương mại điện tử là bài toán xung đột tồn kho (*Race Condition*) khi nhiều khách hàng cùng đặt mua sản phẩm cuối cùng vào cùng một thời điểm.

BrewLite áp dụng kỹ thuật **Khóa lạc quan (Optimistic Locking)** kết hợp cùng **Interactive Transaction (`prisma.$transaction`)**:

- Trên bảng `products`, mỗi dòng sản phẩm có một trường số nguyên `version` với giá trị khởi tạo là `0`.
- Khi giao dịch tạo đơn hàng diễn ra, câu lệnh cập nhật số lượng tồn kho bắt buộc phải thỏa mãn điều kiện phiên bản không bị thay đổi:

```
UPDATE products
SET stock = stock - :quantity,
    version = version + 1
WHERE id = :productId
  AND version = :currentVersion
  AND stock >= :quantity;
```

- Nếu hai giao dịch đồng thời cùng đọc phiên bản $version = 0$, giao dịch thứ nhất thực thi thành công sẽ đẩy phiên bản lên $version = 1$. Giao dịch thứ hai khi cập nhật sẽ nhận số dòng bị ảnh hưởng là 0 (`affected_rows = 0`).
- Khi phát hiện số dòng cập nhật bằng 0, hệ thống ngay lập tức ném lỗi `409 ConflictException` (”Sản phẩm vừa có thay đổi, vui lòng thử lại”), đồng thời tự động rollback toàn bộ giao dịch tạo đơn, đảm bảo tồn kho tuyệt đối không bao giờ bị âm.

6.4 Nghiệp vụ Khuyến mãi và Điểm tích lũy Thành viên

Hệ thống tích hợp bộ đôi động cơ khuyến mãi và khách hàng thân thiết hoạt động nhịp nhàng nhằm gia tăng giá trị đơn hàng:

6.4.1 Cơ chế Mã giảm giá (Promotions Engine)

Hệ thống hỗ trợ cấu hình mã giảm giá đa dạng được quản lý trong bảng `promotions`. Tiêu biểu là mã ưu đãi `WELCOME10`:

- **Mã code:** `WELCOME10`.
- **Tỷ lệ chiết khấu:** Giảm 10% trên tổng giá trị tạm tính của đơn hàng (`total_amount`).
- **Điều kiện áp dụng:** Giá trị đơn hàng tối thiểu (`min_order_amount`) phải đạt từ 100.000 VNĐ trở lên.
- **Mức giảm tối đa:** Giới hạn mức chiết khấu tối đa (`max_discount`) là 50.000 VNĐ.
- Công thức tính:

$$discountAmount = \min\left(totalAmount \times 10\%, 50.000\right)$$

6.4.2 Cơ chế Điểm thưởng Thành viên (Loyalty Points)

- **Tích lũy điểm (Earn Points):** Với mỗi đơn hàng được thanh toán thành công, khách hàng được tích lũy 1 điểm thưởng cho mỗi 10.000 VNĐ giá trị thực trả (`finalAmount`):

$$pointsEarned = \left\lfloor \frac{finalAmount}{10.000} \right\rfloor$$

- **Quy đổi điểm (Redeem Points):** Khách hàng có thể sử dụng điểm tích lũy để trừ trực tiếp vào đơn hàng mới. Giá trị quy đổi là $1 im = 1.000 VN$. Nhằm đảm bảo an toàn doanh thu, hệ thống đặt ra quy tắc cứng: số tiền giảm từ điểm thưởng tối đa không được vượt quá 50% giá trị đơn hàng sau khi đã trừ mã giảm giá.
- Mọi biến động cộng/trừ điểm đều được ghi vết minh bạch trong bảng `loyalty_transactions`.

Chương 7

HIỆN THỰC HÓA VÀ ĐỐI CHIẾU THIẾT KẾ

Chương này trình bày ngắn gọn các minh chứng kỹ thuật nhằm xác nhận rằng toàn bộ các thiết kế Use Case, sơ đồ tương tác và mô hình dữ liệu trong Giai đoạn 1 và Giai đoạn 2 đã được cài đặt đầy đủ vào mã nguồn thực tế của dự án BrewLite.

7.1 Ma trận Ánh xạ Nhiệm vụ GD1 & GD2 sang Mã nguồn

Bảng 7.1 tổng hợp chi tiết mối liên kết giữa các nhiệm vụ thiết kế nghiệp vụ của Giai đoạn 1 và 2 với các tập tin và hàm cụ thể trong mã nguồn Backend (`backend/src`) và Frontend (`frontend/src`).

Bảng 7.1: Ma trận Đối chiếu Thiết kế GD1 & GD2 sang Mã nguồn thực tế

Phân hệ	Nhiệm vụ GD1 & GD2	Hiện thực Backend	Hiện thực Frontend

Auth	Đăng ký tài khoản, Đăng nhập nhận JWT	`auth.controller.ts` (`register`, `login`)
		`auth.service.ts`
			`useAuth.ts`
			`app/login/page.tsx`
			`app/register/page.tsx`

Catalog	Xem danh mục, Lọc món theo loại, Tìm kiếm	`products.controller.ts` (`findAll`, `findCategories`) `products.service.ts`	`useProductCatalog.ts` `app/products/page.tsx`
Detail	Xem chi tiết đồ uống, Chọn size, đường, đá, topping	`products.controller.ts` (`findOne`, `findToppings`) `products.service.ts`	`useProductDetail.ts` `app/products/[id]/page.tsx`
Cart	Quản lý giỏ hàng cục bộ, Thêm/sửa/xóa, Tạm tính	*(FE-Only State Management)*	`useCart.ts` `app/cart/page.tsx`

Order	Tạo đơn hàng, Kiểm tra tồn kho, Tính giá, Áp voucher	`orders.controller.ts` (`createOrder`)
		`orders.service.ts`
		`pricing.service.ts`
		`promotions.service.ts`
			`useCheckout.ts`
			`app/checkout/page.tsx`

Payment	Thanh toán an toàn, Xử lý Idempotency, Gọi Gateway	`payments.controller.ts` (`processPayment`) `payments.service.ts`	`usePayment.ts` `app/payment/[id]/page.tsx`
History	Tra cứu danh sách đơn hàng và trạng thái	`orders.controller.ts` (`findUserOrders`) `orders.service.ts`	`useOrderHistory.ts` `app/orders/page.tsx`

7.2 Minh chứng Kỹ thuật Cốt lõi

Dưới đây là một số đoạn trích mã nguồn tiêu biểu chứng minh tính chính xác của các giải pháp kỹ thuật đã thiết kế:

7.2.1 Cơ chế Mã hóa Bcrypt và Cấp phát Token trong AuthService

```
// backend/src/modules/auth/auth.service.ts
const saltRounds = 10;
const passwordHash = await bcrypt.hash(dto.password, saltRounds);

const user = await this.prisma.user.create({
  data: {
    email: dto.email,
    passwordHash,
    fullName: dto.fullName,
    loyaltyPoints: 0,
  },
});
const { passwordHash: _, ...result } = user;
return result;
```

7.2.2 Interactive Transaction và Khóa Lạc quan trong OrdersService

```
// backend/src/modules/orders/orders.service.ts
return await this.prisma.$transaction(async (tx) => {
  for (const item of dto.items) {
    const product = await tx.product.findUnique({ where: { id: item.
        productId } });
    if (!product || product.stock < item.quantity) {
      throw new BadRequestException(`San pham ${product?.name} khong du ton
          kho`);
    }

    const updated = await tx.product.updateMany({
      where: { id: item.productId, version: product.version },
      data: {
        stock: { decrement: item.quantity },
        version: { increment: 1 },
      },
    });

    if (updated.count === 0) {
      throw new ConflictException('Xung dot ton kho, vui long thu lai');
    }
  }
  // Tiep tuc tao Order va OrderItems trong cung transaction...
});
```

7.2.3 Kiểm tra Bất biến Idempotency trong PaymentsService

```
// backend/src/modules/payments/payments.service.ts
const existing = await this.prisma.payment.findUnique({
  where: { idempotencyKey: key },
});

if (existing) {
  if (existing.status === PaymentStatus.SUCCESS) {
    return existing; // Tra ve ngay ket qua da xu ly
  }
  if (existing.status === PaymentStatus.PENDING) {
    throw new ConflictException('Giao dich dang duoc xu ly, vui long cho');
  }
}
```

Chương 8

ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN

8.1 Đánh giá Kết quả Đạt được

Trải qua hai giai đoạn phân tích, thiết kế và hiện thực hóa (Giai đoạn 1 và Giai đoạn 2), nhóm đã hoàn thành toàn diện các mục tiêu đề ra cho bài tập lớn môn Công nghệ Phần mềm:

- **Chuẩn hóa Thiết kế Hệ thống:** Xây dựng hoàn chỉnh bộ tài liệu kỹ thuật từ sơ đồ kiến trúc thành phần, sơ đồ triển khai Docker, sơ đồ lớp Backend, sơ đồ thực thể liên kết (ERD) 12 bảng cho đến chi tiết Use Case, sơ đồ Hoạt động (Activity) và sơ đồ Tuần tự (Sequence) cho toàn bộ 7 phân hệ chức năng.
- **Hiện thực hóa Đầy đủ 7 Phân hệ Nghiệp vụ:** Cài đặt đồng bộ từ giao diện người dùng Next.js 15 đến hệ thống API NestJS 11 và cơ sở dữ liệu PostgreSQL 16. Mọi tương tác giao diện và gọi dịch vụ đều bám sát thiết kế.
- **Làm chủ các Bài toán Kỹ thuật Nâng cao:** Giải quyết trọn vẹn bài toán máy trạng thái đơn hàng (9 chuyển tiếp), bài toán kiểm soát giao dịch thanh toán bất biến (*Idempotency*) và bài toán kiểm soát xung đột tồn kho bằng khóa lạc quan (*Optimistic Locking*).

8.2 Các Hạn chế Hiện tại

Bên cạnh những kết quả đã đạt được, hệ thống BrewLite vẫn tồn tại một số điểm giới hạn kỹ thuật cần được ghi nhận trung thực:

1. **Cơ chế Retry tự động cho Khóa Lạc quan:** Khi xảy ra xung đột phiên bản giữa các request đặt hàng đồng thời, Backend ném mã lỗi `409 Conflict`. Hiện tại, giao diện người dùng mới dừng ở mức hiển thị thông báo lỗi và yêu cầu khách hàng bấm thử lại thủ công, chưa trang bị cơ chế tự động thử lại ở tầng Client với thuật toán lùi thời gian ngẫu nhiên (*Exponential Backoff with Jitter*).
2. **Giao diện Quầy Pha chế (Barista UI) ngoài phạm vi MVP:** Dù Backend đã hoàn thiện trọn vẹn logic chuyển tiếp trạng thái (`PREPARING`, `READY`, `COMPLETED`), nhưng giao diện web/tablet dành riêng cho nhân viên pha chế tại quầy chưa được hiện thực hóa trong phạm vi bản phát hành MVP này.
3. **Tích hợp Cổng thanh toán Thực tế:** Hệ thống hiện đang sử dụng dịch vụ Mock Payment Gateway để giả lập luồng thanh toán. Việc tích hợp các cổng thanh toán thương mại thực tế tại Việt Nam (như VNPay, MoMo, ZaloPay) đòi hỏi quy trình đăng ký doanh nghiệp và thẩm định pháp lý phức tạp nên được định hướng ở giai đoạn sau.

8.3 Hướng Phát triển trong Tương lai

Nhằm đưa BrewLite trở thành một nền tảng vận hành hoàn chỉnh cho các chuỗi cửa hàng cà phê thương mại, các định hướng phát triển tiếp theo bao gồm:

- **Xây dựng Màn hình Pha chế (Barista POS Display):** Phát triển giao diện chuyên dụng cho quầy bar, cập nhật danh sách đơn hàng theo thời gian thực sử dụng công nghệ Web-Sockets hoặc Server-Sent Events (SSE).
- **Tích hợp Cổng thanh toán VNPay / MoMo:** Kết nối trực tiếp qua Webhook và chuẩn thanh toán mã VietQR, hỗ trợ hoàn tiền tự động khi đơn hàng bị hủy sau khi đã thanh toán.
- **Cơ chế Client-side Auto-Retry:** Nâng cấp lớp thư viện gọi API (API Client) để tự động thử lại các thao tác đặt hàng khi gặp mã lỗi `409 Conflict` mà không gây phiền toái cho trải nghiệm người dùng.
- **Phân hệ Báo cáo và Quản trị Doanh thu (Admin Analytics):** Bổ sung biểu đồ trực quan hóa dữ liệu bán hàng, dự báo nhu cầu nguyên vật liệu và quản lý các chiến dịch khuyến mãi theo mùa.

Chương A

MA TRẬN TRUY VẾT YÊU CẦU (TRACEABILITY MATRIX)

Ma trận truy vết yêu cầu cung cấp khả năng kiểm soát chất lượng từ yêu cầu chức năng (FR), đặc tả Use Case (UC), API Endpoint, bảng cơ sở dữ liệu cho đến các ca kiểm thử (TC) tương ứng.

Bảng A.1: Ma trận Truy vết Toàn diện Yêu cầu - Thiết kế - Cài đặt - Kiểm thử

Mã FR	Mã UC	API Endpoint	Bảng CSDL	Mã Test Case	Màn hình
FR-01	UC-AUTH-01	POST /auth/register	users	TC-AUTH-API-01	/register
	UC-AUTH-02	POST /auth/login	users	TC-AUTH-API-05	/login
FR-02	UC-CAT-01	GET /categories	categories	TC-CAT-API-01	/products
		GET /products	products	TC-CAT-API-02	/products
	UC-DET-01	GET /products/:id	products, sizes	TC-DET-API-01	/products/:id
		GET /toppings	toppings	TC-DET-API-04	/products/:id
FR-03	UC-CART-01	*(Client-side)*	*(LocalStorage)*	TC-CART-UI-01	/cart
FR-04	UC-ORD-01	POST /orders	orders, items, toppings, history	TC-ORD-API-01	/checkout
		GET /orders/:id	orders, items	TC-ORD-API-08	/checkout
FR-05	UC-PAY-01	POST /payments	payments, orders, loyalty, history	TC-PAY-API-01	/payment/:id
		GET /payments/:id	payments	TC-PAY-API-07	/payment/:id
FR-06	UC-HIS-01	GET /orders	orders	TC-HIS-API-01	/orders
		GET /orders/:id	orders, items, status_history	TC-HIS-API-02	/orders/:id

Chương B

DANH MỤC API ENDPOINTS

Bảng B.1 liệt kê toàn bộ các API Endpoints được cài đặt và cung cấp bởi hệ thống Backend NestJS của BrewLite. Tất cả các endpoint đều có tiền tố chuẩn `/api/v1`.

Bảng B.1: Danh mục chi tiết toàn bộ các API Endpoints hệ thống BrewLite

Phương thức	Đường dẫn (Path)	Xác thực	Mô tả chức năng
POST	/api/v1/auth/register	Công khai	Đăng ký tài khoản người dùng mới
POST	/api/v1/auth/login	Công khai	Đăng nhập và nhận mã Bearer JWT
GET	/api/v1/categories	Công khai	Lấy danh sách phân loại danh mục sản phẩm
GET	/api/v1/products	Công khai	Tìm kiếm và lọc danh sách sản phẩm
GET	/api/v1/products/:id	Công khai	Xem chi tiết thông tin và giá theo size sản phẩm
GET	/api/v1/toppings	Công khai	Lấy danh sách các món topping khả dụng
POST	/api/v1/orders	Bearer JWT	Tạo đơn hàng mới từ giỏ hàng, trừ kho, áp voucher
GET	/api/v1/orders/:id	Bearer JWT	Xem chi tiết thông tin và trạng thái đơn hàng
GET	/api/v1/orders	Bearer JWT	Lấy danh sách lịch sử đơn hàng của người dùng
POST	/api/v1/payments	Bearer JWT + Idempotency	Thực hiện thanh toán trực tuyến với khóa bất biến
GET	/api/v1/payments/:id	Bearer JWT	Tra cứu trạng thái giao dịch thanh toán

Mọi phản hồi thành công từ hệ thống đều được chuẩn hóa qua `TransformInterceptor` theo cấu trúc:

```
{
  "success": true,
  "data": { ... },
  "message": "Thong bao thanh cong",
  "timestamp": "2026-10-25T10:00:00.000Z"
}
```

Mọi phản hồi lỗi đều được chuẩn hóa qua `AllExceptionsFilter` theo cấu trúc:

```
{
  "success": false,
  "statusCode": 400,
  "message": "Thong tin loi chi tiet",
  "error": "Bad Request",
  "timestamp": "2026-10-25T10:00:00.000Z",
```

```
  "path": "/api/v1/orders"
}
```

Chương C

TỪ ĐIỂN DỮ LIỆU (DATA DICTIONARY)

Phụ lục này trình bày cấu trúc chi tiết của 12 bảng trong cơ sở dữ liệu PostgreSQL của BrewLite, bám sát tập tin mô hình dữ liệu `backend/prisma/schema.prisma`.

C.1 Bảng `users`

Tên cột	Kiểu dữ liệu	Null	Mặc định	Mô tả / Ràng buộc
`id`	VARCHAR(30)	No	cuid()	Khóa chính định danh người dùng
`email`	VARCHAR(255)	No	-	Địa chỉ email (UNIQUE)
`password_hash`	VARCHAR(255)	No	-	Mật khẩu băm Bcrypt 10 rounds
`full_name`	VARCHAR(100)	Yes	NULL	Họ và tên hiển thị
`loyalty_points`	INTEGER	No	0	Điểm thưởng tích lũy thành viên
`created_at`	TIMESTAMP	No	NOW()	Thời điểm tạo tài khoản
`updated_at`	TIMESTAMP	No	NOW()	Thời điểm cập nhật cuối cùng

C.2 Bảng `categories`

Tên cột	Kiểu dữ liệu	Null	Mặc định	Mô tả / Ràng buộc
`id`	VARCHAR(30)	No	cuid()	Khóa chính
`name`	VARCHAR(100)	No	-	Tên phân loại (UNIQUE)
`slug`	VARCHAR(100)	No	-	Đường dẫn thân thiện (UNIQUE)
`display_order`	INTEGER	No	0	Thứ tự sắp xếp hiển thị tab
`created_at`	TIMESTAMP	No	NOW()	Thời điểm tạo
`updated_at`	TIMESTAMP	No	NOW()	Thời điểm cập nhật

Tên cột	Kiểu dữ liệu	Null	Mặc định	Mô tả / Ràng buộc
id	VARCHAR(30)	No	cuid()	Khóa chính
category_id	VARCHAR(30)	Yes	NULL	Khóa ngoại → categories.id
name	VARCHAR(150)	No	-	Tên đồ uống
base_price	DECIMAL(12,0)	No	-	Đơn giá cơ bản (tiền chẵn VNĐ)
image_url	VARCHAR(500)	Yes	NULL	Đường dẫn hình ảnh
stock	INTEGER	No	0	Số lượng tồn kho khả dụng
version	INTEGER	No	0	Phiên bản Optimistic Locking
is_available	BOOLEAN	No	TRUE	Trạng thái mở bán
created_at	TIMESTAMP	No	NOW()	Thời điểm tạo
updated_at	TIMESTAMP	No	NOW()	Thời điểm cập nhật

Tên cột	Kiểu dữ liệu	Null	Mặc định	Mô tả / Ràng buộc
id	VARCHAR(30)	No	cuid()	Khóa chính
product_id	VARCHAR(30)	No	-	Khóa ngoại → products.id (Cascade)
size	ENUM	No	-	Giá trị: 'S', 'M', 'L'
price_adjustment	DECIMAL(12,0)	No	0	Phụ thu thêm theo size (UNIQUE:[product_id, size])

Tên cột	Kiểu dữ liệu	Null	Mặc định	Mô tả / Ràng buộc
id	VARCHAR(30)	No	cuid()	Khóa chính
name	VARCHAR(100)	No	-	Tên topping (UNIQUE)
price	DECIMAL(12,0)	No	-	Đơn giá topping
stock	INTEGER	No	100	Tồn kho topping
is_available	BOOLEAN	No	TRUE	Trạng thái kinh doanh

Tên cột	Kiểu dữ liệu	Null	Mặc định	Mô tả / Ràng buộc
id	VARCHAR(30)	No	cuid()	Khóa chính
code	VARCHAR(50)	No	-	Mã voucher (UNIQUE)
discount_percent	INTEGER	Yes	NULL	% chiết khấu (ví dụ 10)
discount_amount	DECIMAL(12,0)	Yes	NULL	Tiền chiết khấu cố định
max_discount	DECIMAL(12,0)	Yes	NULL	Mức giảm tối đa (ví dụ 50.000đ)
min_order_amount	DECIMAL(12,0)	No	0	Giá trị đơn hàng tối thiểu
is_active	BOOLEAN	No	TRUE	Trạng thái kích hoạt

Tên cột	Kiểu dữ liệu	Null	Mặc định	Mô tả / Ràng buộc
`id`	VARCHAR(30)	No	cuid()	Khóa chính
`order_number`	SERIAL	No	autoincrement	Mã hiển thị khách hàng (UNIQUE)
`user_id`	VARCHAR(30)	No	-	Khóa ngoại → `users.id` (INDEX)
`promotion_id`	VARCHAR(30)	Yes	NULL	Khóa ngoại → `promotions.id`
`status`	ENUM	No	'PENDING'	Trạng thái đơn (INDEX)
`total_amount`	DECIMAL(12,0)	No	-	Tạm tính trước giảm
`discount_amount`	DECIMAL(12,0)	No	0	Số tiền giảm giá
`final_amount`	DECIMAL(12,0)	No	-	Tiền thực trả
`points_earned`	INTEGER	No	0	Điểm loyalty tích được
`version`	INTEGER	No	0	Khóa lạc quan máy trạng thái

C.3 Bảng `products`

C.4 Bảng `product_size_prices`

C.5 Bảng `toppings`

C.6 Bảng `promotions`

C.7 Bảng `orders`

C.8 Bảng `order_items`

Tên cột	Kiểu dữ liệu	Null	Mặc định	Mô tả / Ràng buộc
`id`	VARCHAR(30)	No	cuid()	Khóa chính
`order_id`	VARCHAR(30)	No	-	Khóa ngoại → `orders.id` (Cascade, INDEX)
`product_id`	VARCHAR(30)	No	-	Khóa ngoại → `products.id`
`size`	ENUM	No	'M'	Kích cỡ: 'S', 'M', 'L'
`quantity`	INTEGER	No	1	Số lượng đặt
`unit_price`	DECIMAL(12,0)	No	-	Đơn giá snapshot tại thời điểm đặt
`line_total`	DECIMAL(12,0)	No	-	Thành tiền của dòng món

Tên cột	Kiểu dữ liệu	Null	Mặc định	Mô tả / Ràng buộc
`id`	VARCHAR(30)	No	cuid()	Khóa chính
`order_item_id`	VARCHAR(30)	No	-	Khóa ngoại → `order_items.id` (Cascade)
`topping_id`	VARCHAR(30)	No	-	Khóa ngoại → `toppings.id`
`price`	DECIMAL(12,0)	No	-	Giá snapshot topping

Tên cột	Kiểu dữ liệu	Null	Mặc định	Mô tả / Ràng buộc
`id`	VARCHAR(30)	No	cuid()	Khóa chính
`order_id`	VARCHAR(30)	No	-	Khóa ngoại → `orders.id` (INDEX)
`idempotency_key`	VARCHAR(100)	No	-	Khóa chống trùng lặp (UNIQUE)
`request_hash`	VARCHAR(64)	Yes	NULL	Mã băm SHA-256 nội dung request
`amount`	DECIMAL(12,0)	No	-	Số tiền thanh toán
`method`	ENUM	No	-	'BANK_CARD' hoặc 'E_WALLET'
`status`	ENUM	No	'PENDING'	'PENDING', 'SUCCESS', 'FAILED'
`transaction_ref`	VARCHAR(100)	Yes	NULL	Mã đối soát từ Gateway

Tên cột	Kiểu dữ liệu	Null	Mặc định	Mô tả / Ràng buộc
`id`	VARCHAR(30)	No	cuid()	Khóa chính
`user_id`	VARCHAR(30)	No	-	Khóa ngoại → `users.id` (INDEX)
`order_id`	VARCHAR(30)	Yes	NULL	Khóa ngoại → `orders.id`
`points`	INTEGER	No	-	Số điểm cộng hoặc trừ
`type`	ENUM	No	-	'EARNED', 'SPENT', 'REFUNDED'
`reason`	VARCHAR(255)	No	-	Lý do biến động điểm

C.9 Bảng `order_item_toppings`

C.10 Bảng `payments`

C.11 Bảng `loyalty_transactions`

C.12 Bảng `order_status_history`

Tên cột	Kiểu dữ liệu	Null	Mặc định	Mô tả / Ràng buộc
`id`	VARCHAR(30)	No	cuid()	Khóa chính
`order_id`	VARCHAR(30)	No	-	Khóa ngoại → `orders.id` (Cascade, INDEX)
`from_status`	ENUM	Yes	NULL	Trạng thái bắt đầu
`to_status`	ENUM	No	-	Trạng thái chuyển đến
`changed_by`	VARCHAR(50)	Yes	NULL	Tác nhân thực hiện (user, barista, system)
`reason`	VARCHAR(255)	Yes	NULL	Lý do chuyển trạng thái
`created_at`	TIMESTAMP	No	NOW()	Thời điểm ghi nhận